

Programming in Python

Maulana Hafidz Ismail

31 Oct 2025

Contents

1	Memulai dengan Python	1
1.1	Tentang Python	1
1.2	Sejarah Pemrograman	1
1.3	Dasar-dasar Pemrograman	1
1.4	Apa itu Pemrograman?	1
1.5	Manfaat Belajar Python	2
1.6	Aplikasi Python	2
1.7	Menjalankan Python di Command Line	2
1.8	Sintaksis Python dan Pentingnya Spasi	2
1.9	Memberi Komentar pada Kode	3
1.9.1	Single-line comments	4
1.9.2	Multi-line comments	4
1.9.3	Inline comments	4
1.10	Pengertian Variabel	4
1.11	Penamaan Variabel	4
1.12	Mengubah dan Menghapus Variabel	5
1.13	Deklarasi Beberapa Variabel	5
1.14	Tipe Data dalam Python	5
1.15	Jenis Tipe Data Utama	5
1.16	Menentukan Tipe Data	6
1.17	Deklarasi Strings	6
1.18	Mengakses dan Mengubah String	7
1.19	Fungsi dan Operasi pada Strings	7
1.20	Contoh Penggunaan	7
1.21	Cheatsheets Tipe Data dan Fungsi	7
1.21.1	Tipe Data	7
1.21.2	Flow Control	7
1.21.3	Indentasi	8
1.21.4	Komentar	8
1.21.5	Built-in Functions	9
1.21.6	Membuat Fungsi	10
1.22	Apa itu Typecasting?	10
1.23	Implicit Type Conversion	10
1.24	Explicit Type Conversion	10
1.25	Fungsi Typecasting Lainnya	11
1.26	Fungsi Input	11
1.27	Fungsi Print	11
1.28	Parameter Tambahan untuk Print	11
1.29	Contoh Penggunaan Input dan Print	11
1.30	Penanganan Tipe Data	12
1.31	Penggabungan String	12
1.32	Melihat Lebih Dalam ke Type Casting	12
1.33	Operator Matematika	14
1.34	Operator Logika	15

1.35	Contoh Penggunaan Operator dalam Pernyataan Kondisional . .	15
1.36	Apa itu Control Flow?	15
1.37	Conditional Statements	16
1.38	Contoh Penggunaan Control Flow	16
1.39	Penggunaan Match Statement	16
1.40	Perbandingan dengan if Statement	17
1.41	Fitur Penting dari Match Statement	17
1.42	For Loop	17
1.43	While Loop	17
1.44	Enumerate Function	18
1.45	Catatan Penting terkait Looping	18
1.46	Pengertian Nested Loops	19
1.47	Cara Kerja Nested Loops	19
1.48	Contoh Penggunaan Nested Loops	19
1.49	Visualisasi Nested Loops	19
1.50	Kompleksitas Waktu	19
2	Dasar Programming dengan Python	21
2.1	Definisi Fungsi	21
2.2	Cara Mendeklarasikan Fungsi	21
2.3	Penggunaan Fungsi	21
2.4	Contoh Praktis	21
2.5	Keuntungan Menggunakan Fungsi	21
2.6	Jenis-jenis Scope	22
2.7	Penjelasan dan Contoh	22
2.8	Apa itu Struktur Data?	23
2.8.1	Mutabilitas dan Imutabilitas	23
2.9	Pengertian List	24
2.10	Deklarasi List	24
2.11	Akses dan Indeks List	24
2.12	List Bersarang	24
2.13	Menambahkan Item ke List	24
2.14	Menghapus Item dari List	25
2.15	Iterasi Melalui List	25
2.16	Apa itu Tuple?	25
2.17	Cara Mendeklarasikan Tuple	25
2.18	Mengakses Elemen dalam Tuple	25
2.19	Menentukan Tipe Tuple	25
2.20	Metode pada Tuple	26
2.21	Perbedaan antara Tuple dan List	26
2.22	Definisi Sets	26
2.23	Operasi Dasar pada Sets	26
2.24	Operasi Matematika pada Sets	26
2.25	Catatan Penting Terkait Set	27
2.26	Apa itu Dictionary?	27
2.27	Keuntungan Menggunakan Dictionary	27

2.28	Cara Mengakses dan Memanipulasi Dictionary	27
2.29	Iterasi Melalui Dictionary	28
2.30	Contoh Pembuatan dan Penggunaan Dictionary	28
2.31	Apa itu args?	28
2.32	Apa itu kwargs?	29
2.33	Struktur data mana yang harus dipilih?	29
2.33.1	Contoh: Daftar Pegawai	29
2.34	Errors	30
2.35	Exceptions	31
2.36	Penanganan Exceptions	31
2.37	Penanganan Exceptions	31
2.38	Mengakses Informasi Exception	32
2.39	Menangani Beberapa Exceptions	32
2.40	File Handling: open dan close	32
2.41	With Statement	32
2.42	Membuat File	33
2.43	Menyisipkan Konten	33
2.44	Mengganti dan Menambahkan Konten	33
2.45	Menangani Kesalahan (Exception)	33
2.46	Metode untuk Membaca File	34
2.47	Jalur File	34
2.48	Contoh Penggunaan Open Statement	35
3	Paradigma Pemrograman	36
3.1	Apa itu Procedural Programming?	36
3.2	Struktur Kode dalam Procedural Programming	36
3.3	Prinsip DRY (Don't Repeat Yourself)	36
3.4	Kelebihan dan Kekurangan Procedural Programming	36
3.5	Pengertian Algoritma	37
3.6	Contoh Penerapan Algoritma	37
3.7	Tipe Algoritma	37
3.8	Refactoring	38
3.9	Mengukur Kinerja Kode	38
3.10	Jenis Time Complexity	38
3.11	Notasi Big-O	39
3.11.1	Apa itu Notasi Big-O	39
3.11.2	Contoh Umum dari Notasi Big-O	40
3.11.3	Ringkasan Singkat	42
3.11.4	Kenapa Notasi Big-O Penting?	44
3.11.5	Menganalisa Kode dengan Notasi Big-O	44
3.12	Pengertian Fungsi	45
3.13	Perbedaan antara Traditional dan Pure Functions	45
3.14	Ciri-ciri Functional Programming	45
3.15	Apa itu Pure Functions?	45
3.16	Contoh dan Modifikasi Fungsi Pure	46
3.17	Manfaat Pure Functions	46

3.18	Apa itu Recursion?	46
3.19	Contoh Penggunaan Rekursi	46
3.20	Kelebihan dan Kekurangan Rekursi	47
3.21	Membalikkan String dalam Python	47
3.22	Fungsi map	48
3.23	Fungsi filter	48
3.24	Perbedaan antara map dan filter	48
3.25	Comprehensions	49
3.25.1	List Comprehension	49
3.25.2	Dictionary Comprehension	50
3.25.3	Set Comprehension	51
3.25.4	Generator Comprehension	51
3.26	Konsep Dasar Object Oriented Programming (OOP)	52
3.27	Komponen Utama OOP	52
3.28	Konsep Kunci dalam OOP	53
3.29	Kilas Balik Konsep OOP	56
3.30	Membuat Class	56
3.31	Menginstansiasi Class	56
3.32	Mengakses Atribut	57
3.33	Membuat dan Mengakses Metode	57
3.34	Catatan Lain Terkait Class	57
3.35	Code Reusability	58
3.36	Metode Khusus dalam Python	58
3.37	Instance Variables	59
3.38	Methods	59
3.39	Konsep Inheritance	59
3.40	Atribut dan Metode	59
3.41	Multiple Inheritance	60
3.42	Multi-level Inheritance	60
3.43	Built-in Functions Terkait Class	61
3.44	Kelas Abstrak (Abstract Class)	62
3.45	Metode Abstrak (Abstract Method)	62
3.46	Sintaks untuk Mendefinisikan Kelas Abstrak dan Metode Abstrak	63
3.46.1	Contoh Implementasi	63
3.47	Konsep Dasar MRO	63
3.47.1	Tipe-tipe Inheritance	63
3.48	Aturan MRO	64
3.49	Algoritma MRO	64
3.50	Menemukan MRO	64
3.51	Eksperimen MRO	65
3.51.1	Contoh 1	65
3.51.2	Contoh 2	65
3.51.3	Contoh 3	66
3.51.4	Contoh 4	67

1 Memulai dengan Python

1.1 Tentang Python

Python dinamai setelah acara TV "Monty Python's Flying Circus" dan bukan dari ular piton. Nama ini dipilih karena dianggap singkat dan misterius. Python digunakan dalam aplikasi yang sering kita gunakan, seperti Instagram, Facebook, Google, dan Spotify. Contoh penggunaan Python dalam machine learning termasuk TensorFlow, yang digunakan oleh Airbnb untuk mengklasifikasikan gambar dan oleh perusahaan kesehatan untuk mengklasifikasikan data MRI. Python dianggap mudah digunakan dan sederhana, dengan banyak pustaka (libraries) yang mendukung pengembangan. Memungkinkan pengembang untuk membuat fitur dengan cepat dan melihat hasilnya lebih cepat.

1.2 Sejarah Pemrograman

- Charles Babbage, pada tahun 1822, menciptakan difference engine untuk mengurangi kesalahan manusia dalam perhitungan.
- Babbage juga mengembangkan analytical engine, yang dianggap sebagai dasar dari komputer modern.
- Teman Babbage, Ada Lovelace, menulis dokumen yang menjelaskan bagaimana analytical engine dapat melakukan urutan perhitungan, mirip dengan apa yang dilakukan oleh program komputer.

1.3 Dasar-dasar Pemrograman

- Komputer hanya memahami binary code, yang terdiri dari dua digit: 0 dan 1. Binary Code: Sistem bilangan yang hanya menggunakan dua angka, 0 (off) dan 1 (on).
- Setiap program yang ditulis harus dikonversi menjadi machine code (kode mesin) agar dapat dipahami oleh komputer. Contoh konversi: Decimal 1 menjadi Binary 1, Decimal 2 menjadi Binary 10.
- Komputer menggunakan transistors untuk merepresentasikan binary code. Transistor: Komponen elektronik yang berfungsi sebagai saklar untuk mengontrol aliran listrik.

1.4 Apa itu Pemrograman?

Pemrograman adalah kemampuan untuk memberikan serangkaian instruksi kepada komputer dalam bahasa yang dapat dipahami.

Pemrograman adalah keterampilan yang dapat diasah dengan latihan. Semakin sering berlatih, semakin baik kemampuan pemrograman seseorang.

Pemrograman juga merupakan keterampilan kreatif, karena ada banyak cara untuk menyelesaikan masalah dengan menulis program.

1.5 Manfaat Belajar Python

- Mudah Dipelajari: Python memiliki sintaks yang mirip dengan bahasa Inggris, sehingga lebih mudah dibaca dan dipahami.
- Produktivitas Tinggi: Dengan Python, pengembang dapat menyelesaikan proyek lebih cepat karena sintaks yang sederhana dan memerlukan lebih sedikit kode dibandingkan dengan bahasa lain seperti C atau Java.

1.6 Aplikasi Python

- Berbagai Platform: Python dapat digunakan di Windows, Mac, dan Linux.
- Bidang Penggunaan: Python banyak digunakan dalam pengembangan web, kecerdasan buatan, machine learning, analisis data, dan aplikasi pemrograman lainnya.

1.7 Menjalankan Python di Command Line

- Python Shell:

Ini adalah cara pertama untuk menjalankan program Python. Shell ini berguna untuk menjalankan dan menguji skrip kecil tanpa perlu membuat file baru dengan ekstensi .PY. Anda dapat mengetikkan potongan kode langsung di shell.

Cukup ketik `python` di command line untuk membuka Python shell, lalu masukkan kode yang ingin dijalankan.

- Menjalankan File Python

Cara kedua adalah menjalankan file Python langsung dari command line. Setiap file dengan ekstensi .PY dapat dijalankan dengan perintah tertentu.

Ketik `python nama_file.py` di command line untuk menjalankan file tersebut.

1.8 Sintaksis Python dan Pentingnya Spasi

Menggunakan `print("Hello")` untuk menghasilkan teks. `print` adalah fungsi yang digunakan untuk menampilkan output ke layar.

```
print("Hello") print("World")
```

Jika mencoba menulis dua pernyataan `print` dalam satu baris tanpa pemisah, akan muncul error `SyntaxError: invalid syntax`. `SyntaxError` adalah kesalahan yang terjadi ketika kode tidak mengikuti aturan sintaks yang benar.

Mengatasi Error:

- Metode Pertama: Memindahkan pernyataan `print` kedua ke baris baru.

```
print("Hello")
print("World")
```

- Metode Kedua: Memisahkan pernyataan dengan titik koma.

```
print("Hello"); print("World")
```

Whitespace adalah karakter kosong (spasi, tab, newline) yang tidak terlihat tetapi mempengaruhi struktur kode. Menambahkan spasi di sekitar operator tidak menyebabkan masalah, tetapi menambahkan baris baru bisa menyebabkan error. Contohnya seperti ini:

```
#any ammount of whitespace on a single line is ok
x      =      1      +      2
```

Dan berikut tidak benar:

```
x = 1
+ 2
```

Indentasi adalah penggunaan spasi atau tab untuk mengatur blok kode, yang menunjukkan hierarki dan struktur. Penting untuk struktur kontrol seperti if statement.

```
if name == "John":
    print(name)
```

Jika indentasi dihapus, akan muncul error `IndentationError: expected an indented block`.

1.9 Memberi Komentar pada Kode

Menambahkan komentar ke kode tidak hanya membantu Anda sebagai pengembang, tetapi juga membantu anggota tim lainnya. Komentar sangat berguna untuk menyegarkan ingatan Anda tentang kode yang mungkin telah Anda tulis berbulan-bulan lalu dan Anda mungkin lupa apa tujuannya. Ada beberapa alasan mengapa Anda perlu menambahkan komentar ke berkas kode. Alasannya antara lain:

- Menjelaskan tujuan kode.
- Memberitahu pengembang bahwa kode tersebut sudah usang.
- Menambahkan komentar TODO untuk pekerjaan yang akan diselesaikan nanti.

Berikut adalah contoh berbagai jenis komentar yang dapat digunakan.

1.9.1 Single-line comments

Penggunaan simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini.

```
# Don't try to Run Me, I'm a comment  
x = 10
```

1.9.2 Multi-line comments

Tanda Kutip Tiga (''' atau '''): Python menggunakan tanda kutip tunggal atau ganda tiga kali untuk membuat komentar multi-baris. Tanda kutip ini biasanya digunakan untuk docstring atau komentar yang mencakup beberapa baris.

1.9.3 Inline comments

Simbol `#` memberi tahu Python untuk mengabaikan semua yang ada setelah titik ini hingga akhir baris saat ini. Hal ini memungkinkan komentar ditempatkan di dalam kode itu sendiri. Komentar sebaris sebaiknya digunakan untuk memberikan informasi penting tentang kode yang ditanganinya.

```
x = 1 # Resetting buffer size
```

Ingatlah untuk selalu memiliki alasan untuk berkomentar, komentar harus memberikan informasi kepada pembaca dan tidak mengganggu.

1.10 Pengertian Variabel

Variabel adalah tempat untuk menyimpan data yang dapat berubah. Dalam Python, Anda dapat mendeklarasikan variabel dengan memberikan nama dan nilai.

Contoh sintaks: `X = 10` (di mana `X` adalah nama variabel dan `10` adalah nilainya).

1.11 Penamaan Variabel

Penting untuk memberikan nama yang bermakna agar mudah dipahami oleh programmer lain.

Dua konvensi penamaan yang umum digunakan:

- **camelCase**: Huruf pertama dari kata pertama kecil, dan huruf pertama dari setiap kata berikutnya besar. Contoh: `myName`.
- **snake_case**: Semua huruf kecil dengan garis bawah sebagai pemisah. Contoh: `my_name`.

1.12 Mengubah dan Menghapus Variabel

Anda dapat mengubah nilai variabel yang sudah dideklarasikan dengan cara mendeklarasikannya kembali.

```
a = 10
print(a) # Output: 10
a = 5
print(a) # Output: 5
```

Untuk menghapus variabel, gunakan perintah del.

```
del a
print(a) # Akan menghasilkan error: NameError: name 'a' is
        not defined
```

1.13 Deklarasi Beberapa Variabel

Anda dapat mendeklarasikan beberapa variabel sekaligus.

```
a, b, c = 1, 2, 3
print(a, b, c) # Output: 1 2 3
```

1.14 Tipe Data dalam Python

Tipe data adalah atribut yang terkait dengan data yang memberi tahu sistem komputer bagaimana menginterpretasikan nilainya.

Tipe Data memastikan data dikumpulkan dalam format yang diinginkan dan nilai setiap properti sesuai harapan.

1.15 Jenis Tipe Data Utama

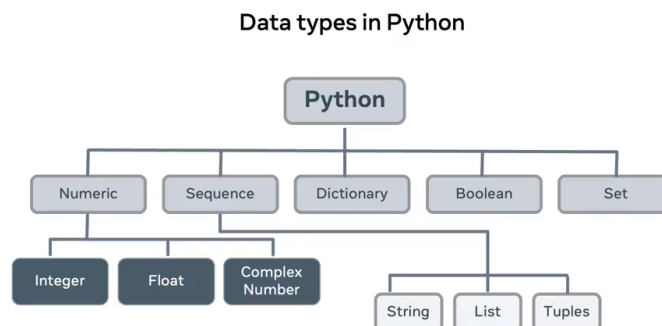


Figure 1: Pohon Tipe Data

- Numeric: Tipe data ini mencakup:

- Integer: Angka bulat tanpa desimal, misalnya 10 atau -10.
- Float: Angka dengan desimal, misalnya 10.5 atau 6.7.
- Complex: Angka kompleks yang terdiri dari bagian nyata dan imajiner, misalnya $a = 10 + 10j$.
- Sequence: Tipe data yang berisi satu atau lebih tipe dalam urutan yang teratur, termasuk:
 - String: Sekuens karakter yang diapit oleh tanda kutip, misalnya "Hello".
 - List: Sekuens yang dapat berisi berbagai tipe, diapit oleh tanda kurung siku, misalnya [1, 2, 3].
 - Tuple: Mirip dengan list, tetapi tidak dapat diubah (immutable), diapit oleh tanda kurung, misalnya (1, 2, 3).
- Dictionary: Menyimpan data dalam struktur objek key-value. Contoh:

```
ed = {'a': 22, 'b': 44.4}
```

- Boolean: Tipe data yang hanya memiliki dua nilai, yaitu True atau False. Contoh:

```
is_valid = True
```

- Set: Koleksi tidak teratur dari nilai yang tidak diulang. Contoh:

```
example_set = {1, 2, 3, 4}
```

1.16 Menentukan Tipe Data

Fungsi `type()`: Digunakan untuk menentukan tipe data dari variabel. Contoh:

```
a = 10
print(type(a)) # Output: <class 'int'>
```

1.17 Deklarasi Strings

String adalah sekuens karakter yang dikelilingi oleh tanda kutip tunggal (') atau ganda ("). Contoh sintaks: `a = 'hello'` atau `b = "hello"`.

Encoding adalah proses mengubah karakter menjadi bentuk yang dapat dipahami komputer. Python menggunakan `Unicode` untuk encoding.

1.18 Mengakses dan Mengubah String

- Indexing: Setiap karakter dalam string dapat diakses menggunakan indeks. Python menggunakan zero-indexing, artinya indeks dimulai dari 0. Contoh: `name[0]` mengakses karakter pertama.
- Mengubah Nilai: Anda dapat mengubah nilai string dengan cara menetapkan nilai baru ke variabel. Contoh: `name = 'Paul'`.

1.19 Fungsi dan Operasi pada Strings

- `len()`: Fungsi untuk memeriksa panjang string. Contoh sintaks: `len(name)` akan mengembalikan jumlah karakter dalam string.
- Concatenation: Menggabungkan dua string menggunakan operator `+`. Contoh: `a + b` akan menggabungkan string `a` dan `b`.

1.20 Contoh Penggunaan

- Single Line String: `a = 'this is a single line'`
- Multi-Line String: Untuk membuat string yang panjang, gunakan backslash (`\`) di akhir baris. Contoh:

```
b = 'this_is_a_multi-line_string_example_\nthat_continues_here.'
```

- Output: Menggunakan `print()` untuk menampilkan string. Contoh: `print(a)` akan mencetak nilai dari `a`.

1.21 Cheatsheets Tipe Data dan Fungsi

Berikut ini adalah panduan singkat tentang jenis data dalam Python.

1.21.1 Tipe Data

Tipe Data	Meaning	Example
string	Text	'Hello', "Testing 123"
integer	Numbers	-5, 4, 3, 2, 0
float	Decimals	2.4, 5.2, 1000.00

1.21.2 Flow Control

Operator Perbandingan

Operator	Meaning	Example
==	Equals	a == b
!=	Not Equal	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or Equal to	a <= b
>=	Greater than or Equal to	a >= b

1.21.3 Indentasi

Indentasi digunakan untuk mendefinisikan blok kode dalam Python. Berbeda dengan banyak bahasa pemrograman yang menggunakan kurung kurawal untuk mengelompokkan kode, Python menggunakan indentasi.

Indentasi wajib digunakan dalam Python dan digunakan untuk menunjukkan pernyataan mana yang termasuk dalam loop, fungsi, kondisional, dan struktur lainnya.

```
if x > 0: # Colon indicates the start of the block
    print("Positive") # Indented
else:
    print("Non-positive") # Indented
```

1.21.4 Komentar

Menempatkan simbol # di depan teks yang ingin Anda jadikan komentar akan membuat Python mengabaikan semua teks dari titik tersebut hingga akhir baris saat ini.

Triple Quotes (''' or """): Python menggunakan tanda kutip tunggal atau ganda bertiga untuk membuat komentar **multi-baris**. Tanda kutip ini umumnya digunakan untuk docstrings atau komentar yang mencakup beberapa baris.

Simbol # akan membuat Python mengabaikan semua teks mulai dari titik tersebut hingga akhir baris saat ini, sehingga komentar inline dapat dibuat dengan cara ini.

Kolon (:): Digunakan untuk menandai awal blok kode, seperti dalam perulangan, kondisi, dan definisi fungsi.

```
# Single Line comment

# This is a multiline comment
# which can be used for long comments

x = 1 # assigns value of 1 to x

if x > 0: # Colon starts the block
    print("Positive")
```

1.21.5 Built-in Functions

- `print()`

Fungsi ini mencari perangkat output default, terminal Anda, dan menampilkan nilai yang diteruskan kepadanya.

```
print("Hello")
```

- `input()`

Fungsi ini mencari perangkat input default, yaitu keyboard Anda, dan menangkap nilainya. Nilai ini kemudian dapat ditugaskan atau digunakan.

```
print("Where do you live?")  
location = input()  
print("So you live in " + location)
```

Catatan: Blok kode di atas akan menampilkan prompt kepada pengguna dengan pertanyaan "Di mana Anda tinggal?". Pengguna kemudian akan memasukkan respons dan menekan tombol Return untuk melanjutkan eksekusi kode yang tersisa. Respons yang dimasukkan oleh pengguna disimpan, dikembalikan sebagai output, atau dapat ditugaskan ke variabel tertentu tergantung pada konteks penggunaannya.

- `len()`

Fungsi ini mengembalikan panjang atau jumlah elemen yang terdapat dalam struktur yang diterapkan padanya. Struktur tersebut dapat berupa string, array, daftar, tuple, kamus, atau urutan apa pun.

```
len("Hello")  
5
```

- `str()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi String.

```
str(55)  
'55'
```

- `int()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan bulat.

```
int('75')  
75
```

- `float()`

Fungsi ini dapat digunakan untuk mengubah nilai yang diberikan menjadi bilangan float.

```
some_int = 10
float(some_int)
10.0
```

1.21.6 Membuat Fungsi

Fungsi dalam Python memerlukan kata kunci untuk mendefinisikannya: ‘def’ diikuti oleh identifikasi (nama), yang membentuk tanda tangan fungsi. Tubuh fungsi berisi kode yang akan dijalankan saat fungsi dipanggil.

```
def say_hello():
    return "Hello there!"

# With parameters
def say_hello(you):
    return "Hello " + you
```

1.22 Apa itu Typecasting?

Typecasting adalah proses mengubah satu tipe data menjadi tipe data lain. Dalam Python, ada dua jenis konversi: implicit dan explicit.

1.23 Implicit Type Conversion

Implicit Conversion : Dilakukan secara otomatis oleh compiler Python untuk mencegah kehilangan data. Contoh: mengubah int menjadi float jika nilai yang dimasukkan adalah desimal.

Penting untuk dicatat bahwa Python hanya dapat mengonversi nilai jika tipe data tersebut kompatibel. Misalnya, int dan float kompatibel, tetapi String dan int tidak.

1.24 Explicit Type Conversion

Explicit Conversion : Dilakukan oleh pengembang menggunakan fungsi yang disediakan oleh Python. Beberapa

- `str()`: Mengubah tipe data menjadi string. Sintaks: `str(value)`.
- `int()`: Mengubah tipe data menjadi integer. Sintaks: `int(value)`.
- `float()`: Mengubah tipe data menjadi float. Sintaks: `float(value)`.

1.25 Fungsi Typecasting Lainnya

- `ord()`: Mengembalikan integer yang mewakili karakter unicode yang mendasarinya.
- `hex()`: Mengonversi integer menjadi string heksadesimal.
- `oct()`: Mengambil integer dan mengembalikan string yang mewakili angka oktal.
- `tuple()`, `set()`, `list()`, `dict()`: Fungsi lain untuk mengonversi tipe data.

1.26 Fungsi Input

Fungsi `input()` digunakan untuk mendapatkan data dari pengguna.

Sintaks: `input("Prompt")`

- Contoh: `email = input("Please enter your email address: ")`
- Penjelasan: Pengguna akan diminta untuk memasukkan alamat email, yang kemudian disimpan dalam variabel `email`.

1.27 Fungsi Print

Fungsi `print()` digunakan untuk menampilkan output ke layar.

Sintaks: `print(value1, value2, ..., sep=' ', end='\n')`

- Contoh: `print("Hello", "World", sep=" ")`
- Penjelasan: Menampilkan "Hello, World" dengan koma sebagai pemisah.

1.28 Parameter Tambahan untuk Print

- **sep**: Menentukan pemisah antara objek yang dicetak.
- **end**: Menentukan apa yang dicetak di akhir output (default adalah newline).
- **file**: Menentukan di mana nilai dicetak (default adalah STD out).
- **flush**: Boolean untuk mengatur apakah buffer harus dibersihkan.

1.29 Contoh Penggunaan Input dan Print

Mengambil dua angka dari pengguna dan mencetak hasil penjumlahannya:

```
num1 = int(input("Please enter the first number: "))
num2 = int(input("Please enter the second number: "))
print("The sum is:", num1 + num2)
```

Penjelasan: Menggunakan `int()` untuk mengonversi input menjadi integer sebelum melakukan penjumlahan.

1.30 Penanganan Tipe Data

Input dari fungsi `input()` selalu berupa string.

Sintaks: `type(variable)`

- Contoh: `print(type(num1))`
- Penjelasan: Menampilkan tipe data dari variabel `num1`, yang akan menunjukkan bahwa itu adalah string jika tidak dikonversi.

1.31 Penggabungan String

Menggabungkan dua atau lebih string menggunakan operator `+`.

Contoh:

```
first_name = input("Please enter your first name: ")
last_name = input("Please enter your last name: ")
print("Hello, " + first_name + " " + last_name)
```

Penjelasan: Menampilkan sapaan dengan nama depan dan belakang yang dimasukkan pengguna.

1.32 Melihat Lebih Dalam ke Type Casting

Ada beberapa situasi di mana tipe data dari suatu nilai perlu diubah menjadi tipe data lain.

Proses ini dikenal sebagai casting tipe.

Cara lain yang lebih informal untuk menyebutnya adalah "konversi tipe data".

Contoh paling sederhana dari konversi data dapat dilihat pada perbandingan berikut:

```
print(10 == 10)
```

Dalam potongan kode di atas, saya bertanya kepada Python apakah angka 10 sama dengan angka 10, dan saya akan mendapatkan nilai boolean `True` yang dicetak, yang mengonfirmasi bahwa memang keduanya sama.

Bagaimana jika saya melakukan ini?

```
print(10 == 10.00)
```

Sekali lagi, nilai boolean `True` ditampilkan di layar.

Sekarang, Anda mungkin keberatan bahwa, ya, 10 secara teknis tidak sama dengan 10.0 - karena, bisa dibilang, angka pertama adalah bilangan bulat, dan angka kedua adalah bilangan desimal. Anda benar - meskipun ini adalah angka yang sama, mereka memiliki tipe data yang berbeda.

Namun, Python di sini melakukan apa yang disebut "konversi tipe implisit".

Untuk memahami bagaimana ini bekerja, saya akan sedikit mengubah contoh sebelumnya. Alih-alih meminta Python untuk membandingkan kedua angka dan mengembalikan apakah keduanya sama atau tidak, saya akan meminta Python untuk menampilkan hasil penjumlahan kedua angka tersebut.

```
print(10 + 10.0)
```

Hasil cetak adalah 20.0.

Hasil ini memungkinkan saya menyimpulkan bahwa saat Python menjalankan operasi yang melibatkan bilangan bulat dan bilangan desimal, ia secara implisit mengubah tipe bilangan bulat menjadi bilangan desimal, lalu menyelesaikan operasi tersebut.

Untuk benar-benar menekankan poin ini, saya dapat memperluas contoh sebelumnya dengan menggunakan fungsi `type()`:

```
print(type(10 + 10.0))
```

Hasilnya adalah `<class 'float'>`, jadi ini mengonfirmasi kesimpulan saya.

Sekarang saya akan menunjukkan kepada Anda program kecil dalam Python yang bekerja dengan angka:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Ketika saya menjalankan program ini, misalnya, saya dapat memasukkan nilai angka pertama sebagai 5, dan nilai angka kedua juga sebagai 5, dengan harapan nilai `user_sum` yang dicetak akan menjadi 10.

Namun, ketika saya melakukan hal itu, angka 55 yang dicetak.

Mengapa ini tidak berfungsi?

Jawabannya cukup sederhana: segala sesuatu yang diketik oleh pengguna oleh Python dikonversi menjadi tipe data string.

Ini berarti, meskipun saya mengetik angka ke dalam dua input tersebut, yang disimpan dalam `user_num_1` dan `user_num_2` sebenarnya adalah string.

Secara efektif, ini sama saja dengan jika saya hanya melakukan ini:

```
user_num_1 = "5"
user_num_2 = "5"
user_sum = user_num_1 + user_num_2
print(user_sum)
```

Kali ini, hasil cetak dari `user_sum` masih "55", tetapi hal ini tidak melanjutkan.

Untuk membuat kode Python saya berfungsi sesuai yang saya inginkan, saya perlu melakukan konversi tipe secara eksplisit.

Dengan kata lain, saya harus mengubah nilai "5" menjadi nilai 5.

Berikut adalah kode saya yang telah diperbarui:

```
user_num_1 = input('First number is: ')
user_num_2 = input('Second number is: ')
user_sum = float(user_num_1) + float(user_num_2)
print(user_sum)
```

Apa yang saya lakukan di sini adalah memastikan bahwa program saya dapat menangani input berupa bilangan float dan tetap berfungsi dengan benar.

Dengan kata lain, saya memastikan bahwa jika pengguna memasukkan nilai float 5.5 sebagai angka pertama dan kedua saat menjalankan kode di atas, outputnya tidak akan menimbulkan kesalahan. Sebaliknya, outputnya akan menjadi 11.0 seperti yang diharapkan.

Bagaimana jika saya memutuskan untuk menampilkan beberapa kata kepada pengguna, memberitahu mereka apa yang terjadi?

Ini adalah upaya untuk melakukannya:

```
num_1 = input('First number is: ')
num_2 = input('Second number is: ')
user_sum = float(num_1) + float(num_2)
print("The sum of " + num_1 + " and " + num_2 + " is " +
      user_sum)
```

Jika saya menjalankan kode di atas, akan muncul kesalahan berikut:

```
Traceback (most recent call last):
  File "<string>", line 4, in <module>
TypeError: hanya dapat menggabungkan str (bukan "float")
dengan str
```

Artinya, saya tidak dapat menggabungkan string dan float seperti itu. Dengan kata lain, meskipun konversi tipe implisit Python berfungsi saat menggunakan operator + pada string dan integer, hal ini tidak berlaku untuk string dan float kecuali Anda secara eksplisit mengonversi float menjadi string.

Perlu diperhatikan juga, Anda juga tidak dapat menggabungkan string dan integer. Hal ini terjadi karena Python menerapkan keamanan tipe, artinya ia memerlukan instruksi eksplisit tentang cara menggabungkan tipe data yang berbeda dalam kasus seperti string dengan integer.

Solusi untuk ini adalah melakukan konversi tipe secara eksplisit, sebagai berikut.

```
n1 = input('First number is: ')
n2 = input('Second number is: ')
user_sum = float(n1) + float(n2)
print("The sum of " + n1 + " and " + n2 + " is " + str(
      user_sum))
```

Kali ini, hasilnya adalah: Jumlah dari 5,5 dan 5,5 adalah 11,0.

1.33 Operator Matematika

- Addition (+): Digunakan untuk menjumlahkan angka. Contoh sintaks: 2 + 3 menghasilkan 5.
- Subtraction (-): Digunakan untuk mengurangi angka. Contoh sintaks: 3 - 2 menghasilkan 1.

- Division (/): Digunakan untuk membagi angka. Contoh sintaks: `35 / 5` menghasilkan 7.0.
- Multiplication (*): Digunakan untuk mengalikan angka. Contoh sintaks: `7 * 4` menghasilkan 28.

1.34 Operator Logika

- AND: Memeriksa apakah semua kondisi benar. Contoh: `a < 5 and a < 10` hanya benar jika a lebih besar dari 5 dan kurang dari 10.
- OR: Memeriksa apakah setidaknya satu kondisi benar. Contoh: `a < 5 or b < 10` benar jika salah satu dari a atau b memenuhi kondisi.
- NOT: Mengembalikan nilai false jika hasilnya true. Contoh: `not (a < 5)` akan benar jika a tidak lebih besar dari 5.

1.35 Contoh Penggunaan Operator dalam Pernyataan Kondisional

Operator biasanya digunakan dengan pernyataan kondisional untuk mengontrol alur program. Contoh: Untuk memberikan diskon di restoran, Anda bisa memeriksa apakah pelanggan adalah anggota program loyalitas dan telah menghabiskan lebih dari \$100.

```
if in_loyalty_program and total_bill > 100:
    total_bill = total_bill - (total_bill * 0.1)
```

Contoh Lain:

- Mathematical Operators:
 - `print(2 + 2)` akan menampilkan 4.
 - `print(35 / 5)` akan menampilkan 7.0.
- Logical Operators:
 - `a = True; b = True; if a and b: print("all true")` hanya akan mencetak "all true" jika kedua a dan b adalah true.

1.36 Apa itu Control Flow?

Control Flow merupakan urutan di mana instruksi dalam program dieksekusi. Program akan mengambil tindakan yang berbeda berdasarkan keputusan yang dibuat.

Dalam Python, ada dua jenis control flow:

- Conditional Statements: Seperti `if`, `else`, dan `elif`.
- Loops: Seperti `for` loop dan `while` loop.

1.37 Conditional Statements

- if: Menyatakan bahwa jika kondisi terpenuhi, maka fungsi akan dijalankan.

```
if condition:
    # code to execute
```

- else: Menjalankan fungsi ketika semua kondisi tidak terpenuhi.

```
else:
    # code to execute if the condition is false
```

- elif: Menyatakan bahwa jika kondisi sebelumnya tidak terpenuhi, coba kondisi ini.

```
elif another_condition:
    # code to execute if this condition is true
```

1.38 Contoh Penggunaan Control Flow

Dalam contoh restoran, jika total tagihan (`bill_total`) lebih dari 100, maka diskon diterapkan.

```
bill_total = 114
discount1 = 10
if bill_total > 100:
    bill_total = bill_total - discount1
print("Total bill:", str(bill_total))
```

Untuk menambahkan diskon untuk tagihan di atas 200, gunakan elif,

```
elif bill_total > 200:
    discount2 = 20
    bill_total = bill_total - discount2
```

1.39 Penggunaan Match Statement

Match Statement diperkenalkan di Python versi 3.10, match statement memungkinkan penulisan kode yang lebih bersih dan mudah dibaca dibandingkan dengan if statement.

```
match variable:
    case value1:
        # aksi jika variable sama dengan value1
    case value2:
        # aksi jika variable sama dengan value2
    case _:
        # aksi default jika tidak ada yang cocok
```

1.40 Perbandingan dengan if Statement

If Statement mengharuskan penulisan banyak kondisi dengan if, elif, dan else, yang dapat membuat kode menjadi kompleks.

```
if http_status == 200:
    print("Success")
elif http_status == 404:
    print("Not Found")
else:
    print("Unknown")
```

1.41 Fitur Penting dari Match Statement

- Menggabungkan Beberapa Kondisi: Anda dapat menggunakan operator `or` untuk menggabungkan beberapa kondisi dalam match statement.
- Default Case: Menggunakan `case _` untuk menangani situasi di mana tidak ada nilai yang cocok, mirip dengan else dalam if statement.
- Fleksibilitas: Memberikan cara yang lebih bersih untuk menangani banyak kondisi dibandingkan dengan if statement.

1.42 For Loop

For loop adalah struktur kontrol yang digunakan untuk iterasi melalui sequence (seperti string atau array).

Sintaks:

```
for item in sequence:
    # kode yang akan dijalankan
```

Contoh:

```
str = "looping"
for item in str:
    print(item)
```

1.43 While Loop

While loop adalah struktur kontrol yang menjalankan blok kode selama kondisi tertentu.

Sintaks:

```
while condition:
    # kode yang akan dijalankan
```

Contoh:

```
count = 0
favorites = ["cake", "ice_cream", "cookies", "brownies", "
pie"]
while count < len(favorites):
    print("I_like_this_dessert:", favorites[count])
    count += 1
```

1.44 Enumerate Function

Fungsi `enumerate()` digunakan dalam for loop untuk mendapatkan indeks dan nilai item dalam urutan.

Sintaks:

```
for index, item in enumerate(sequence):
    # kode yang akan dijalankan
```

Menggunakan `enumerate()` untuk mencetak indeks dan nilai:

```
for idx, item in enumerate(favorites):
    print(idx, item)
```

Ini akan mencetak indeks dan nama dessert dari array `favorites`.

1.45 Catatan Penting terkait Looping

- Infinite Loop: Jika kondisi dalam while loop tidak pernah menjadi false, loop akan terus berjalan tanpa henti. Pastikan untuk mengincrement counter agar tidak terjadi infinite loop.
- Indexing: Dalam Python, indeks dimulai dari 0, jadi item pertama memiliki indeks 0 dan item terakhir memiliki indeks n-1, di mana n adalah jumlah item dalam urutan.
- Pernyataan **break** digunakan untuk menghentikan loop, yang pada gilirannya juga menghentikan kondisi else.
- Sama seperti perintah break, perintah **continue** dapat digunakan untuk mengontrol iterasi loop. Perbedaan utamanya adalah perintah continue memungkinkan Anda untuk melewati bagian tertentu dari loop, tetapi kemudian melanjutkan ke bagian sisanya.
- Pernyataan **pass** berfungsi sebagai placeholder, memungkinkan Anda untuk menyertakan blok kode kosong tanpa menyebabkan kesalahan sintaks. Pernyataan ini tidak melakukan apa-apa dan memungkinkan program untuk melanjutkan eksekusi secara normal.

1.46 Pengertian Nested Loops

Nested Loop adalah struktur pengulangan di mana satu loop (inner loop) berada di dalam loop lainnya (outer loop). Ini memungkinkan iterasi melalui beberapa koleksi data secara bersamaan.

1.47 Cara Kerja Nested Loops

- Outer Loop: Loop yang menjalankan iterasi pertama. Setelah itu, ia akan masuk ke inner loop.
- Inner Loop: Loop yang berjalan hingga batas yang ditentukan. Setelah selesai, kontrol kembali ke outer loop untuk iterasi berikutnya.

1.48 Contoh Penggunaan Nested Loops

Misalkan ada dua list dari angka satu hingga sembilan. Dengan menggunakan nested loops, kita dapat menghitung total iterasi:

```
count = 0
for i in range(1, 10): # Outer loop
    for j in range(1, 10): # Inner loop
        count += 1
print(count) # Output: 81
```

Penjelasan: Outer loop berjalan 9 kali, dan inner loop berjalan 9 kali untuk setiap iterasi outer loop, sehingga total iterasi adalah $9 * 9 = 81$.

1.49 Visualisasi Nested Loops

Untuk memahami lebih baik, kita bisa mencetak hasil dari setiap iterasi:

```
for i in range(10): # Outer loop
    for j in range(10): # Inner loop
        print(0, end=' ')
    print() # New line after inner loop
```

Penjelasan: Ini akan mencetak grid 2D dari angka nol.

1.50 Kompleksitas Waktu

Time Complexity mengacu pada waktu yang dibutuhkan untuk menjalankan kode. Semakin besar array, semakin lama waktu yang dibutuhkan.

Contoh: Jika outer loop memiliki range 100 dan inner loop 1000, waktu eksekusi akan meningkat.

```
import time
start_time = time.time()
# Code to measure
```

```
print("Execution_time:", round(time.time() - start_time, 2)  
    )
```

2 Dasar Programming dengan Python

2.1 Definisi Fungsi

Fungsi adalah sekumpulan instruksi yang dapat digunakan kembali untuk melakukan tugas tertentu. Dalam Python, fungsi dideklarasikan dengan kata kunci `def`.

2.2 Cara Mendeklarasikan Fungsi

Sintaks untuk mendeklarasikan fungsi:

```
def function_name(parameters):  
    # kode yang akan dijalankan
```

Contoh:

```
def sum(x, y):  
    return x + y
```

2.3 Penggunaan Fungsi

- Fungsi dapat menerima parameter dan mengembalikan nilai. Contoh fungsi `print()` yang digunakan untuk menampilkan output ke layar.
- Fungsi `input()` digunakan untuk menerima input dari pengguna.

2.4 Contoh Praktis

- Menghitung jumlah pajak dari total tagihan:

```
def calculate_tax(bill, tax_rate):  
    return bill * tax_rate / 100
```

- Memanggil fungsi:

```
total_tax = calculate_tax(175, 15)  
print(total_tax) # Output: 26.25
```

2.5 Keuntungan Menggunakan Fungsi

Fungsi memungkinkan kode menjadi lebih modular dan reusable. Jika ada perubahan, Anda hanya perlu memperbarui fungsi tersebut, dan semua bagian kode yang memanggil fungsi itu akan mendapatkan pembaruan.

2.6 Jenis-jenis Scope

- **Local Scope:** Variabel yang dideklarasikan dalam fungsi dan hanya dapat diakses di dalam fungsi tersebut.
- **Enclosing Scope:** Variabel yang dideklarasikan dalam fungsi luar dan dapat diakses oleh fungsi dalamnya.
- **Global Scope:** Variabel yang dideklarasikan di luar semua fungsi dan dapat diakses dari mana saja dalam kode.
- **Built-in Scope:** Variabel dan fungsi yang sudah tersedia di Python, seperti `print()` dan `len()`.

2.7 Penjelasan dan Contoh

LEGB merupakan akronim untuk Local, Enclosing, Global, dan Built-in, yang menunjukkan urutan pencarian variabel.

- Contoh Global Scope:

```
my_global = 10
def fn1():
    local_v = 5
    print("Access to global:", my_global)
fn1() # Output: Access to global: 10
print(local_v) # Akan menghasilkan error
```

Penjelasan: Variabel `my_global` dapat diakses di dalam fungsi `fn1`, tetapi `local_v` tidak dapat diakses di luar fungsi.

- Contoh Enclosing Scope:

```
def fn1():
    enclosed_v = 8
    def fn2():
        print("Access to enclosed:", enclosed_v)
    fn2()
fn1() # Output: Access to enclosed: 8
```

Penjelasan: Fungsi `fn2` dapat mengakses variabel `enclosed_v` yang dideklarasikan di dalam `fn1`.

- Built-in Scope:

Merupakan fungsi dan object yang sudah ada di Python, seperti `print()` yang dapat diakses dari semua level scope.

Namun **reserved keyword** seperti `def`, `for`, `if`, etc tidak termasuk built-in scope.

2.8 Apa itu Struktur Data?

Sejauh ini, Anda hanya menyimpan sedikit data dalam variabel. Ini adalah bilangan bulat, Boolean atau string.

Tetapi apa yang terjadi jika Anda perlu bekerja dengan informasi yang lebih kompleks, seperti kumpulan data seperti daftar orang atau daftar perusahaan?

Struktur data dirancang untuk tujuan ini.

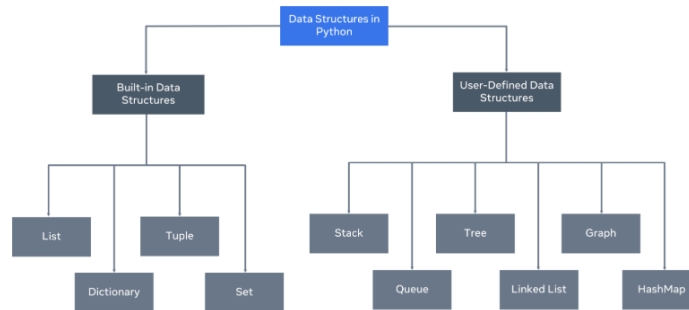


Figure 2:

Struktur data memungkinkan Anda untuk mengorganisir dan mengatur data Anda agar dapat melakukan operasi pada data tersebut. Python memiliki struktur data bawaan berikut: list, dictionary, tuple, dan set. Semua struktur data ini dianggap sebagai struktur data non-primitif, artinya mereka diklasifikasikan sebagai objek.

Selain struktur data bawaan, Python memungkinkan pengguna untuk membuat struktur data mereka sendiri. Struktur data seperti Tumpukan (Stacks), Antrian (Queues), dan Pohon (Trees) semuanya dapat dibuat oleh pengguna.

Setiap struktur data dapat dirancang untuk memecahkan masalah tertentu atau mengoptimalkan solusi yang ada agar menjadi jauh lebih efisien.

2.8.1 Mutabilitas dan Imutabilitas

Struktur data dapat bersifat mutable atau immutable. Pertanyaan selanjutnya yang mungkin Anda tanyakan adalah, apa itu mutabilitas? Mutabilitas mengacu pada data di dalam struktur data yang dapat dimodifikasi. Misalnya, Anda dapat mengubah, memperbarui, atau menghapus data saat diperlukan. List adalah contoh struktur data yang dapat diubah. Lawan dari mutabilitas adalah tidak dapat diubah. Struktur data yang tidak dapat diubah tidak akan mengizinkan modifikasi setelah data ditetapkan. Tuple adalah contoh struktur data yang tidak dapat diubah.

2.9 Pengertian List

List adalah sebuah urutan dari satu atau lebih tipe data yang berbeda atau serupa. Dalam Python, list adalah array dinamis yang dapat menyimpan tipe data apa pun.

2.10 Deklarasi List

Untuk membuat list, gunakan sintaks:

```
list1 = [1, 2, 3, 4, 5] # List berisi angka
list2 = ["A", "B", "C"] # List berisi string
list3 = ["Hello", 10, True, 3.14] # List dengan berbagai
    tipe data
```

2.11 Akses dan Indeks List

List diindeks mulai dari 0. Untuk mengakses item, gunakan sintaks:

```
list1[2] # Mengakses item ketiga, hasilnya 3
```

2.12 List Bersarang

Nested List: List yang berisi list lain. Contoh:

```
list4 = [1, [2, 3, 4], 5, 6] # List dengan nested list
```

2.13 Menambahkan Item ke List

- `insert()`: Menambahkan item pada indeks tertentu.

```
list1.insert(5, 6) # Menambahkan 6 di akhir list
```

- `append()`: Menambahkan item di akhir list

```
list1.append(6) # Menambahkan 6 di akhir list
```

- `extend()`: Menambahkan beberapa item sekaligus

```
list1.extend([6, 7, 8, 9]) # Menambahkan 6, 7, 8, 9
```

2.14 Menghapus Item dari List

- `pop()`: Menghapus item berdasarkan indeks.

```
list1.pop(4) # Menghapus item kelima
```

- `del`: Menghapus item berdasarkan indeks.

```
del list1[2] # Menghapus item ketiga
```

2.15 Iterasi Melalui List

Untuk mengakses semua item dalam list, gunakan loop:

```
for x in list1:  
    print(x) # Mencetak semua nilai dalam list
```

2.16 Apa itu Tuple?

Tuple adalah struktur data yang dapat menyimpan berbagai jenis data dalam satu variabel. Contoh: Tuple dapat berisi integer, string, double, dan boolean.

2.17 Cara Mendeklarasikan Tuple

Untuk mendeklarasikan tuple, gunakan tanda kurung:

```
my_tuple = (1, "string", 4.5, True)
```

Anda juga dapat mendeklarasikan tuple tanpa tanda kurung, tetapi penggunaan tanda kurung adalah praktik terbaik.

2.18 Mengakses Elemen dalam Tuple

Untuk mengakses elemen dalam tuple, gunakan indeks:

```
print(my_tuple[1]) # Mengakses elemen dengan indeks 1
```

Indeks dimulai dari 0, jadi `my_tuple[1]` akan mengembalikan "string".

2.19 Menentukan Tipe Tuple

Untuk mengetahui tipe dari tuple, gunakan fungsi `type()`

```
print(type(my_tuple)) # Mengembalikan <class 'tuple'>
```

2.20 Metode pada Tuple

- `count()`: Menghitung jumlah kemunculan nilai dalam tuple.

```
my_tuple.count("string") # Mengembalikan 1
```

- `index()`: Mengembalikan indeks dari nilai yang dicari.

```
my_tuple.index(4.5) # Mengembalikan 2
```

2.21 Perbedaan antara Tuple dan List

Nilai dalam tuple tidak dapat diubah setelah dideklarasikan. Jika Anda mencoba mengubah nilai, akan muncul error:

```
my_tuple[0] = 5 # Akan menghasilkan TypeError
```

2.22 Definisi Sets

Set adalah sequence yang tidak mengizinkan nilai duplikat dan tidak memiliki urutan.

2.23 Operasi Dasar pada Sets

- Menambahkan Elemen:
Sintaks: `set_a.add(6)` Definisi: Menambahkan elemen baru ke dalam set.
- Menghapus Elemen:
Sintaks: `set_a.remove(2)` atau `set_a.discard(2)` Definisi: Menghapus elemen dari set. `remove` akan menghasilkan error jika elemen tidak ada, sedangkan `discard` tidak.

2.24 Operasi Matematika pada Sets

- Union:
Sintaks: `set_a.union(set_b)` atau `set_a | set_b` Definisi: Menggabungkan dua set dan menghilangkan nilai duplikat.
- Intersection:
Sintaks: `set_a.intersection(set_b)` atau `set_a & set_b` Definisi: Mengambil elemen yang ada di kedua set.
- Difference:
Sintaks: `set_a.difference(set_b)` atau `set_a - set_b` Definisi: Mengambil elemen yang hanya ada di `set_a` dan tidak ada di `set_b`.

- Symmetrical Difference:

Sintaks: `set_a.symmetric_difference(set_b)` atau `set_a ^ set_b`

Definisi: Mengambil elemen yang ada di salah satu set tetapi tidak di kedua set.

2.25 Catatan Penting Terkait Set

- Tidak Terurut: Set tidak memiliki indeks, sehingga tidak bisa diakses dengan cara seperti `set_a[0]`. Jika dicoba, akan menghasilkan error.
- Koleksi Unik: Set hanya menyimpan elemen unik, tidak ada duplikasi.

2.26 Apa itu Dictionary?

Dictionary adalah struktur data yang menyimpan data dalam bentuk pasangan kunci (key) dan nilai (value). Mirip dengan kamus, di mana Anda mencari kata berdasarkan huruf pertama.

Key-Value Pair: Setiap item dalam dictionary terdiri dari kunci dan nilai yang terkait. Contoh: `{"1": "coffee"}`.

2.27 Keuntungan Menggunakan Dictionary

- Akses Cepat: Anda dapat mengakses nilai langsung menggunakan kunci, tanpa perlu mencari index seperti pada list. Ini membuatnya lebih cepat dan fleksibel.
- Mutable: Nilai dalam dictionary dapat diubah setelah dideklarasikan. Misalnya, Anda dapat mengubah nilai dari "tea" menjadi "mint tea".

2.28 Cara Mengakses dan Memanipulasi Dictionary

- Mengakses Nilai: Untuk mengakses nilai, gunakan sintaks berikut:

```
sample_dictionary = {1: "coffee", 2: "tea"}
print(sample_dictionary[1]) # Output: coffee
```

- Mengupdate Nilai: Untuk mengubah nilai, gunakan sintaks:

```
sample_dictionary[2] = "mint_tea"
```

- Menghapus Item: Untuk menghapus item, gunakan fungsi del:

```
del sample_dictionary[3] # Menghapus item dengan kunci
3
```

2.29 Iterasi Melalui Dictionary

Anda dapat menggunakan beberapa metode untuk iterasi, seperti:

- Standard Iteration: Menggunakan loop for untuk mengakses kunci.
- Items Function: Menggunakan `items()` untuk mendapatkan pasangan kunci-nilai.
- Values Function: Menggunakan `values()` untuk mendapatkan semua nilai.

2.30 Contoh Pembuatan dan Penggunaan Dictionary

- Deklarasi Dictionary:

```
my_d = {1: "test", "name": "Jim"}
```

- Mengakses Kunci:

```
print(my_d[1])    # Output: test
print(my_d["name"]) # Output: Jim
```

- Menambahkan Kunci Baru:

```
my_d[2] = "Test_2"
```

- Mengupdate Kunci:

```
my_d[1] = "not_a_test"
```

- Menghapus Kunci:

```
del my_d[1]
```

2.31 Apa itu args?

`args` adalah cara untuk mengirimkan sejumlah argumen non-keyword ke dalam fungsi. Sintaks:

```
def function_name(*args):
    # kode untuk memproses args
```

Contoh:

```
def sum_of(*args):
    total = 0
    for x in args:
        total += x
    return total
```

Jika kita memanggil `sum_of(4, 5, 6)`, hasilnya adalah 15.

2.32 Apa itu kwargs?

kwargs adalah cara untuk mengirimkan argumen keyword ke dalam fungsi, yang memungkinkan kita untuk menggunakan nama parameter. Sintaks:

```
def function_name(**kwargs):  
    # kode untuk memproses kwargs
```

Contoh:

```
def calculate_bill(**kwargs):  
    total = 0  
    for item, price in kwargs.items():  
        total += price  
    return round(total, 2)
```

2.33 Struktur data mana yang harus dipilih?

Bagian yang paling menantang bagi pengembang pemula adalah memahami struktur data mana yang paling sesuai dengan solusi yang dibutuhkan. Setiap struktur data menawarkan pendekatan yang berbeda dalam menyimpan, mengakses, dan memperbarui informasi yang disimpan di dalamnya. Ada banyak faktor yang perlu dipertimbangkan, termasuk ukuran, kecepatan, dan kinerja. Cara terbaik untuk memahami mana yang lebih sesuai adalah melalui contoh.

2.33.1 Contoh: Daftar Pegawai

Dalam contoh ini, terdapat daftar karyawan yang bekerja di sebuah restoran. Anda perlu dapat menemukan seorang karyawan berdasarkan ID karyawan mereka, ID numerik berbasis integer. Fungsi `get_employee` berisi loop for untuk mengiterasi daftar karyawan dan mengembalikan objek karyawan jika ID-nya cocok. Berikut adalah pemilihan struktur data yang salah

```
employee_list = [(12345, "John", "Kitchen"), (12458, "Paul",  
    "House_Floor")]  
  
def get_employee(id):  
    for employee in employee_list:  
        if employee[0] == id:  
            return {"id": employee[0], "name": employee[1],  
                "department": employee[2]}  
  
print(get_employee(12458))
```

Dalam kode ini, `employee_list` adalah daftar tuple. Kode ini berjalan dengan baik dan akan mengembalikan pengguna Paul, karena ID-nya, 12458, cocok. Tantangannya muncul ketika daftar menjadi lebih besar.

Alih-alih dua karyawan, Anda mungkin memiliki 2000 atau bahkan 20.000. Kode harus mengiterasi daftar secara berurutan hingga angka yang dicari cocok.

Anda dapat mengoptimalkan kode dengan membagi pencarian, tetapi bahkan dengan ini, kinerjanya masih kalah dibandingkan dengan struktur data lain, seperti kamus.

```
employee_dict = {
    12345: {
        "id": "12345",
        "name": "John",
        "department": "Kitchen"
    },
    12458: {
        "id": "12458",
        "name": "Paul",
        "department": "House_Floor"
    }
}

def get_employee_from_dict(id):
    return employee_dict[id];

print(get_employee_from_dict(12458));
```

Perhatikan bahwa, dalam blok kode ini, jika Anda mengubah struktur data menjadi kamus, Anda dapat menemukan karyawan tersebut. Perbedaan utamanya adalah Anda tidak perlu lagi mengiterasi daftar untuk menemukannya. Jika daftar membesar menjadi ukuran yang jauh lebih besar, waktu pencarian untuk menemukan karyawan tetap sama.

Ini adalah contoh yang baik tentang cara memilih struktur data yang tepat untuk solusi.

Keduanya bekerja dengan baik, tetapi pertukaran yang perlu dipertimbangkan adalah antara waktu dan skala. Solusi pertama akan bekerja dengan baik untuk jumlah data yang kecil, tetapi kinerjanya menurun seiring dengan pertumbuhan data.

Solusi kedua lebih cocok untuk jumlah data yang besar karena strukturnya memungkinkan waktu pencarian konstan, sehingga jumlah data yang besar dapat diakses dengan kecepatan konstan.

2.34 Errors

Error adalah kesalahan dalam kode yang biasanya disebabkan oleh kesalahan manusia, seperti kesalahan pengetikan.

Contoh:

- Syntax Error:

Kesalahan yang terjadi ketika kode tidak mengikuti aturan sintaksis Python. Misalnya, tidak menambahkan titik dua (:) di akhir kondisi.

```
if condition
    # kode
```

Kesalahan ini biasanya ditandai oleh IDE dan menunjukkan lokasi kesalahan.

- Indentation Error:

Kesalahan yang terjadi ketika indentasi tidak konsisten.

```
def function():
    print("Hello")
print("World") # Jika tidak diindentasikan dengan benar
```

Kesalahan ini menunjukkan bahwa indentasi tidak sesuai dengan yang diharapkan.

2.35 Exceptions

Exceptions adalah kesalahan yang terjadi saat eksekusi kode dan perlu ditangani oleh pengembang. Contoh:

- ZeroDivisionError: Terjadi ketika mencoba membagi angka dengan nol.

```
result = 5 / 0 # Ini akan menyebabkan ZeroDivisionError
```

Kesalahan ini menunjukkan bahwa operasi matematika tidak valid.

2.36 Penanganan Exceptions

Pengembang harus menangani exceptions untuk mencegah aplikasi gagal.

Contoh: Menggunakan blok `try` dan `except` untuk menangani exceptions.

```
try:
    result = 5 / 0
except ZeroDivisionError:
    print("Tidak bisa dibagi dengan nol.")
```

Blok `try` mencoba menjalankan kode, dan jika terjadi exception, blok `except` akan menangani kesalahan tersebut.

2.37 Penanganan Exceptions

- `try`: Blok kode yang akan dieksekusi. Jika terjadi kesalahan, eksekusi akan berpindah ke blok `except`.
- `except`: Blok kode yang akan dijalankan jika terjadi exception. Ini memungkinkan kita untuk memberikan pesan kesalahan yang lebih ramah pengguna.

2.38 Mengakses Informasi Exception

- `as e`: Menggunakan alias untuk exception yang terjadi, sehingga kita bisa mencetak informasi lebih lanjut.
- `e.__class__`: Mengakses kelas dari exception yang terjadi untuk memberikan informasi lebih spesifik.

2.39 Menangani Beberapa Exceptions

Chaining except: Kita bisa menambahkan beberapa pernyataan except untuk menangani berbagai jenis exceptions.

```
try:
    result = divide_by(40, 0)
except ZeroDivisionError as e:
    print("Division by zero:", e)
except Exception as e:
    print("An error occurred:", e)
```

2.40 File Handling: open dan close

- `open`: Digunakan untuk membuka, membaca, dan menulis file. Memerlukan dua argumen: nama file dan mode.
 - `r`: Membuka file untuk dibaca dalam format teks (Default).
 - `rb`: Membuka file untuk dibaca dalam format biner.
 - `r+`: Membuka file untuk membaca dan menulis.
 - `w`: Membuka file untuk menulis (akan menimpa file yang ada).
 - `a`: Membuka file untuk menambahkan data.
- `close`: Digunakan untuk menutup koneksi file yang terbuka. Tidak memerlukan argumen.
- Membuka file untuk membaca:

```
file = open("test.txt", "r")
data = file.readline()
print(data)
file.close()
```

2.41 With Statement

Pernyataan `with open` adalah cara yang lebih baik untuk membuka file karena secara otomatis menutup file setelah selesai digunakan. Ini juga lebih baik dalam menangani kesalahan.

```
with open("test.txt", "r") as file:
    data = file.readline()
    print(data)
```

2.42 Membuat File

Fungsi `open()` digunakan untuk membuat file baru. Sintaksnya:

```
with open("newfile.txt", "w") as file:
```

`open()` membuka file dengan mode tertentu. Mode "w" berarti write (tulis), yang **akan membuat file baru atau menimpa file yang sudah ada.**

2.43 Menyisipkan Konten

- `write()`: Metode ini digunakan untuk menulis teks ke dalam file. Sintaksnya:

```
file.write("This is a new file created.")
```

`write()` menambahkan string ke file yang dibuka.

- `writelines()`: Metode ini digunakan untuk menulis beberapa baris ke dalam file. Sintaksnya:

```
file.writelines(["This is the first line.\n", "This is the second line."])
```

`writelines()` menerima daftar string dan menulisnya ke file. Untuk memisahkan baris, gunakan `\n` untuk membuat baris baru.

2.44 Mengganti dan Menambahkan Konten

Mode Append untuk menambahkan konten tanpa menimpa file yang ada, gunakan mode "a". Sintaksnya:

```
with open("newfile.txt", "a") as file:
```

Mode "a" (append) memungkinkan penambahan konten ke akhir file yang sudah ada.

2.45 Menangani Kesalahan (Exception)

`try` dan `except` digunakan untuk menangani kesalahan saat membuat file. Sintaksnya:

```
try:
    with open("sample/newfile.txt", "w") as file:
except FileNotFoundError as e:
    print(f"Error: {e}")
```

2.46 Metode untuk Membaca File

- **read:**

Mengembalikan seluruh isi file sebagai string. Anda juga dapat memberikan argumen integer untuk mengembalikan jumlah karakter tertentu.

```
file.read(size)
```

Mengambil semua karakter dari file atau sejumlah karakter yang ditentukan.

- **readline:**

Mengembalikan satu baris dari file sebagai string. Anda juga dapat memberikan argumen integer untuk mengembalikan sejumlah karakter dari baris tersebut.

```
file.readline(size)
```

Mengambil satu baris dari file, atau sejumlah karakter dari baris pertama jika argumen diberikan.

- **readlines:**

Membaca seluruh isi file dan mengembalikannya dalam bentuk daftar yang terurut.

```
file.readlines()
```

Mengambil semua baris dari file dan mengembalikannya sebagai daftar, memungkinkan iterasi atau pemilihan baris tertentu.

2.47 Jalur File

- **Absolute Path:**

Mengandung informasi lengkap untuk menemukan file, termasuk label drive atau tanda "/" di depan.

Definisi: Jalur yang menunjukkan lokasi file secara lengkap, terlepas dari direktori saat ini.

- Relative Path:

Tidak mengandung referensi ke direktori root dan biasanya relatif terhadap file yang memanggil. Definisi: Jalur yang hanya mencakup informasi yang diperlukan untuk menemukan file dalam direktori kerja saat ini.

2.48 Contoh Penggunaan Open Statement

- Membuka file dengan mode baca:

```
with open('sample.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

Menggunakan `with open` untuk membuka file, yang secara otomatis menutup file setelah selesai digunakan.

- Menggunakan `read` untuk mencetak isi file:

```
print(file.read())
```

Mencetak seluruh isi file.

- Menggunakan `readline` untuk mencetak baris pertama:

```
print(file.readline())  
Mencetak hanya baris pertama dari file.
```

- Menggunakan `readlines` untuk mendapatkan daftar baris:

```
lines = file.readlines()  
for line in lines:  
    print(line)
```

Mengambil semua baris sebagai daftar dan mencetaknya satu per satu.

3 Paradigma Pemrograman

3.1 Apa itu Procedural Programming?

Procedural Programming adalah paradigma pemrograman yang menyusun kode menjadi prosedur atau subrutin yang dapat dieksekusi secara berurutan. Ini mirip dengan menulis instruksi langkah demi langkah.

3.2 Struktur Kode dalam Procedural Programming

Kode disusun dalam bentuk fungsi yang dapat menerima argumen dan mengembalikan nilai. Contoh sintaks:

```
def sum(a, b):  
    return a + b
```

Fungsi adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil berulang kali dengan argumen yang berbeda.

3.3 Prinsip DRY (Don't Repeat Yourself)

DRY adalah prinsip yang menyarankan untuk menghindari pengulangan kode. Misalnya, alih-alih menulis kode yang sama untuk menambahkan dua angka, kita dapat menggunakan fungsi. Contoh:

```
def add_numbers(num1, num2):  
    return num1 + num2
```

3.4 Kelebihan dan Kekurangan Procedural Programming

Kelebihan:

- Mudah dipahami untuk pemula.
- Prosedur dapat digunakan kembali di bagian lain dari kode.
- Kode lebih terstruktur dan mudah dipahami.

Kekurangan:

- Sulit untuk dipelihara dan diperluas.
- Tidak selalu mencerminkan objek dunia nyata.
- Data dapat terekspos di seluruh program.

3.5 Pengertian Algoritma

Algoritma adalah serangkaian langkah yang diikuti untuk menyelesaikan suatu masalah. Contohnya, mengikuti resep untuk membuat omelet.

Ada berbagai pendekatan untuk hal ini. Beberapa orang lebih suka menulis pseudocode, yaitu sintaksis mirip bahasa Inggris yang menyerupai kode, untuk menjelaskan masalah dalam serangkaian langkah. Pendekatan lain adalah menggunakan Flowchart yang memberikan representasi grafis dari serangkaian langkah tersebut. Flowchart ini mengikuti alur logis algoritma dan menunjukkan berbagai keputusan yang perlu diambil untuk memecahkan masalah tersebut.

- Input: Bahan-bahan yang diperlukan untuk membuat omelet.
- Output: Hasil akhir, yaitu omelet yang sudah jadi.

3.6 Contoh Penerapan Algoritma

- Dalam pemrograman, algoritma digunakan untuk menyelesaikan berbagai masalah, dari yang sederhana hingga yang kompleks.
- Contoh algoritma yang dijelaskan adalah untuk memeriksa apakah sebuah kata adalah palindrome. Palindrome adalah kata yang dapat dibaca sama dari depan dan belakang, seperti "racecar".

3.7 Tipe Algoritma

- Recursion
Rekursi merujuk pada metode atau fungsi yang memanggil dirinya sendiri. Teknik ini digunakan untuk menyelesaikan masalah dengan memecahnya menjadi sub-masalah.
- Divide dan Conquer
Ini terdiri dari dua bagian. Bagian pertama adalah memecah masalah menjadi sub-masalah yang lebih kecil, dan bagian kedua adalah menyelesaikan solusi akhir
- Dynamic Programming
Ini terutama digunakan untuk masalah optimasi. Mirip dengan algoritma divide and conquer, metode ini membagi masalah menjadi submasalah
Programming dinamis adalah teknik algoritmik yang terutama digunakan untuk masalah optimasi. Metode ini bekerja dengan membagi masalah menjadi submasalah yang lebih kecil dan tumpang tindih, menyelesaikan setiap submasalah sekali, dan menyimpan hasilnya untuk digunakan kembali. Hal ini menghindari perhitungan berulang dan membuat solusi menjadi lebih efisien. Dua sifat kunci yang membuat pemrograman dinamis

dapat diterapkan adalah submasalah yang tumpang tindih (masalah-masalah kecil yang sama diselesaikan berulang kali) dan struktur optimal (solusi optimal dari masalah utama dapat dibangun dari solusi optimal submasalahnya). Contoh umum termasuk deret Fibonacci, algoritma jalur terpendek (seperti Bellman-Ford), dan masalah ransel.

- Greedy Algorithm

Greedy Algorithm menemukan solusi terbaik pada setiap langkah, bukan dengan mendekati optimasi secara global.

Greedy Algorithm membangun solusi secara bertahap, selalu memilih opsi yang terlihat terbaik pada langkah saat ini. Ia membuat pilihan optimal secara lokal dengan harapan pilihan tersebut akan mengarah pada solusi optimal secara global. Greedy Algorithm sederhana dan efisien, tetapi hanya berfungsi dengan benar jika masalah memiliki sifat greedy-choice property (optimum global dapat dicapai dengan memilih optimum lokal) dan optimal substructure. Contohnya termasuk pemilihan aktivitas, pengkodean Huffman, algoritma Kruskal dan Prim untuk pohon rentang minimum, dan algoritma Dijkstra untuk jalur terpendek.

3.8 Refactoring

- Definisi: Proses menulis ulang atau mengubah kode untuk membuatnya lebih mudah dikelola atau lebih efisien.
- Pentingnya: Refactoring adalah bagian standar dari siklus pengembangan perangkat lunak.

3.9 Mengukur Kinerja Kode

- Waktu dan Ruang: Kode diukur berdasarkan waktu (berapa lama waktu yang dibutuhkan) dan ruang (berapa banyak memori yang digunakan).
- Big O Notation: Digunakan untuk mengukur efisiensi algoritma dalam hal waktu dan ruang.

3.10 Jenis Time Complexity

- Constant Time ($O(1)$)
 - Definisi: Algoritma yang selalu berjalan dalam waktu dan ruang yang sama, terlepas dari ukuran input.
 - Contoh: Mengambil nilai dari dictionary menggunakan kunci.
 - Sintaks: Tidak ada sintaks khusus, hanya akses langsung ke nilai.
- Linear Time ($O(n)$)
 - Definisi: Waktu eksekusi bertambah sebanding dengan ukuran input.

- Contoh: Mencari elemen dalam array.
- Sintaks: Tidak ada sintaks khusus, tetapi iterasi melalui array.
- Logarithmic Time ($O(\log n)$)
 - Definisi: Waktu eksekusi berkurang secara signifikan dengan setiap iterasi.
 - Contoh: Binary search yang membagi daftar menjadi dua setiap kali.
 - Sintaks: Tidak ada sintaks khusus, tetapi menggunakan metode pembagian.
- Quadratic Time ($O(n^2)$)
 - Definisi: Waktu eksekusi meningkat secara kuadratik dengan ukuran input.
 - Contoh: Nested loops yang mengiterasi setiap elemen dalam daftar.
 - Sintaks:

```
for i in range(n):
    for j in range(n):
        # kode
```
- Exponential Time ($O(2^n)$)
 - Definisi: Waktu eksekusi berlipat ganda dengan setiap iterasi.
 - Contoh: Fibonacci sequence.
 - Sintaks: Tidak ada sintaks khusus, tetapi algoritma yang mengulangi perhitungan.

3.11 Notasi Big-O

Notasi Big O adalah konsep dasar dalam ilmu komputer dan pemrograman yang membantu Anda menganalisis dan menggambarkan efisiensi algoritma. Notasi ini menyediakan cara baku untuk mengekspresikan bagaimana waktu eksekusi atau penggunaan sumber daya algoritma meningkat seiring dengan bertambahnya ukuran data masukan.

3.11.1 Apa itu Notasi Big-O

Bayangkan Anda sedang memasak pasta dan ingin menentukan berapa lama waktu yang dibutuhkan untuk merebus air dalam panci. Anda mungkin mempertimbangkan faktor-faktor seperti ukuran panci, daya kompor, dan jumlah air yang perlu dipanaskan. Demikian pula, dalam ilmu komputer, algoritma dianalisis berdasarkan efisiensinya saat menangani berbagai ukuran data masukan.

Notasi Big O adalah notasi matematis yang menggambarkan batas atas atau skenario terburuk untuk kompleksitas waktu suatu algoritma. Notasi ini membantu kita menjawab pertanyaan seperti:

- How does the runtime of an algorithm change as the input data gets larger?
- How does an algorithm scale with increased input size?

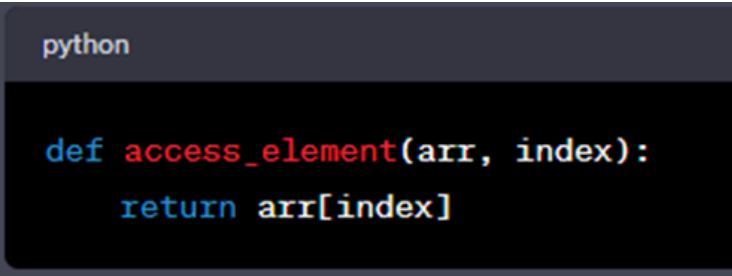
Notasi Big O ditulis sebagai " $O(f(n))$," di mana " $f(n)$ " adalah fungsi yang mewakili hubungan antara ukuran input (biasanya dilambangkan sebagai " n ") dan waktu eksekusi algoritma atau penggunaan sumber daya.

3.11.2 Contoh Umum dari Notasi Big-O

- $O(1)$ - Constant Time

Dalam algoritma dengan kompleksitas waktu konstan, waktu eksekusi tidak bergantung pada ukuran data masukan. Waktu eksekusi tetap konstan, menjadikannya skenario yang paling efisien.

Contoh: Mengakses elemen dalam array berdasarkan indeksnya.



```
python

def access_element(arr, index):
    return arr[index]
```

Figure 3:

Terlepas dari seberapa besar ukuran array, mengakses elemen berdasarkan indeksnya membutuhkan waktu yang sama. Waktu eksekusi bersifat konstan, dan kita menandainya sebagai $O(1)$.

- $O(n)$ - Linear Time

Algoritma dengan kompleksitas waktu linier memiliki waktu eksekusi yang tumbuh secara linier seiring dengan ukuran data masukan. Artinya, jika ukuran data masukan berlipat ganda, waktu eksekusi juga berlipat ganda.

Contoh: Mencari nilai tertentu dalam daftar yang tidak terurut.

```
python

def linear_search(arr, target):
    for item in arr:
        if item == target:
            return True
    return False
```

Figure 4:

Seiring dengan bertambahnya ukuran daftar (arr), jumlah iterasi yang dilakukan oleh loop juga bertambah secara linier. Oleh karena itu, algoritma ini memiliki kompleksitas waktu $O(n)$.

- $O(n^2)$ - Quadratic Time

Algoritma dengan kompleksitas waktu kuadrat memiliki waktu eksekusi yang bertambah seiring dengan kuadrat ukuran input. Seiring dengan bertambahnya ukuran data input, waktu eksekusi bertambah secara kuadrat.

Contoh: Bubble Sort, algoritma pengurutan yang sederhana.

```
python

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Figure 5:

Algoritma Bubble Sort memiliki kompleksitas waktu $O(n^2)$. Seiring dengan bertambahnya ukuran daftar input (arr), jumlah perbandingan dan pertukaran meningkat secara kuadrat.

- $O(\log n)$ - Logarithmic Time

Algoritma dengan kompleksitas waktu logaritmik memiliki waktu eksekusi yang tumbuh secara logaritmik seiring dengan ukuran data masukan. Kompleksitas waktu logaritmik dianggap sangat efisien.

Contoh: Pencarian biner dalam daftar yang terurut.

```
python

def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Figure 6:

Pencarian biner secara drastis mengurangi waktu pencarian seiring dengan bertambahnya ukuran daftar yang terurut (arr). Algoritma ini memiliki kompleksitas waktu $O(\log n)$.

3.11.3 Ringkasan Singkat

- Paling Cepat

$O(1)$ - **Waktu Konstan** : Sangat cepat! Kecepatan algoritma tidak bergantung pada jumlah data yang Anda miliki. Ini seperti mencari buku favorit Anda di rak buku yang tertata rapi – waktu yang dibutuhkan sama, baik Anda memiliki 10 buku atau 1.000 buku.

- Cukup Cepat

$O(\log n)$ - **Waktu Logaritmik** : Masih cukup cepat! Waktunya bertambah secara perlahan seiring penambahan data. Bayangkan seperti mencari nama dalam buku telepon dengan terus membaginya menjadi dua bagian – semakin cepat meskipun buku teleponnya semakin besar.

- Umum

$O(n)$ - **Waktu Linear** : Kecepatan yang memadai! Jika Anda memiliki dua kali lipat data, waktu yang dibutuhkan juga sekitar dua kali lipat. Ini seperti memeriksa daftar nama satu per satu untuk menemukan kecocokan.

- Lebih Lambat

$O(n \log n)$ - **Waktu Linier-Logaritmik** : Lebih cepat dari waktu kuadratik tetapi lebih lambat dari waktu linier. Mirip dengan menyortir tumpukan kartu dengan cepat menggunakan teknik cerdas.

Contoh kompleksitas waktu $O(n \log n)$ adalah algoritma Merge Sort, yang membagi sebuah array menjadi bagian-bagian yang lebih kecil, menyortirnya, dan kemudian menggabungkannya kembali.

- Lebih Lambat Lagi

$O(n^2)$ - **Waktu Kuadrat** : Semakin lambat seiring penambahan data. Seperti memeriksa setiap kombinasi item dalam daftar satu sama lain, tidak ideal untuk daftar besar.

- Cukup Lambat

$O(2^n)$ - **Waktu Eksponensial** : Sekarang kita berbicara tentang kecepatan yang sangat lambat! Waktu komputasi meningkat dengan sangat cepat seiring penambahan data. Bayangkan sebuah teka-teki di mana Anda harus mencoba setiap kombinasi yang mungkin, bahkan untuk teka-teki kecil sekalipun, prosesnya sangat lambat.

- Sangat Lambat

$O(n!)$ - **Waktu Faktorial** : Yang paling lambat di antara semuanya! Ini seperti memecahkan teka-teki kompleks di mana jumlah kemungkinan susunan meledak saat Anda menambahkan lebih banyak potongan. Praktis tidak dapat digunakan untuk masalah besar.

Contoh kompleksitas waktu $O(n!)$ adalah menghasilkan semua kemungkinan susunan (permutasi) dari daftar item, seperti menemukan semua kemungkinan susunan tempat duduk untuk sekelompok orang.

3.11.4 Kenapa Notasi Big-O Penting?

Notasi Big O sangat penting untuk beberapa alasan:

- Perbandingan Algoritma: Notasi ini memungkinkan kita untuk membandingkan algoritma secara objektif dan memilih yang paling efisien untuk tugas tertentu.
- Optimasi Kinerja: Memahami Notasi Big O membantu mengidentifikasi titik lemah dalam kode dan mengoptimalkan algoritma untuk kinerja yang lebih baik.
- Skalabilitas: Algoritma yang efisien sangat penting seiring dengan pertumbuhan aplikasi dan ukuran data.
- Pengelolaan Sumber Daya: Dalam lingkungan dengan sumber daya terbatas, seperti sistem tertanam, memilih algoritma efisien sangat penting.
- Wawancara Koding: Notasi Big O sering diuji dalam wawancara teknis dan tantangan koding, menunjukkan kemampuan Anda untuk menganalisis dan mengoptimalkan algoritma.

3.11.5 Menganalisa Kode dengan Notasi Big-O

Untuk menganalisis kode menggunakan notasi Big O, ikuti langkah-langkah berikut:

- Identifikasi Ukuran Masukan: Tentukan apa yang diwakili oleh "n" dalam kode Anda, yang sering kali berkaitan dengan ukuran data masukan.
- Identifikasi Perulangan dan Iterasi: Cari perulangan dalam kode Anda, karena perulangan sering kali menjadi faktor utama yang memengaruhi kompleksitas waktu.
- Hitung Operasi di Dalam Perulangan: Hitung jumlah operasi di dalam setiap perulangan yang bergantung pada ukuran masukan "n."
- Gabungkan Kompleksitas: Jika Anda memiliki loop bersarang, kalikan kompleksitasnya untuk menentukan kompleksitas waktu keseluruhan.
- Pilih Istilah Dominan: Dalam kasus kompleksitas gabungan, fokus pada istilah dengan laju pertumbuhan tertinggi, karena istilah tersebut akan mendominasi kompleksitas waktu keseluruhan.
- Sederhanakan: Sederhanakan ekspresi sebanyak mungkin dengan menghilangkan faktor konstan.

Dengan mengikuti langkah-langkah ini, Anda dapat menentukan kompleksitas waktu suatu algoritma dan memahami bagaimana kinerjanya akan berubah seiring dengan bertambahnya ukuran input.

Secara ringkas, notasi Big O adalah konsep dasar dalam ilmu komputer yang membantu kita menganalisis dan membandingkan efisiensi algoritma. Dengan memahami dasar-dasarnya dan menerapkannya pada kode, Anda dapat membuat keputusan yang terinformasi tentang pemilihan dan optimasi algoritma, memastikan program Anda berjalan dengan efisien, bahkan saat ukuran data bertambah.

3.12 Pengertian Fungsi

- Fungsi: Blok kode yang menerima input, memprosesnya, dan menghasilkan output.
- Traditional Function: Fungsi yang dapat mengakses dan memodifikasi variabel di global state.
- Pure Function: Fungsi yang selalu memberikan hasil yang sama untuk input yang sama dan tidak mengubah data di luar fungsi.

3.13 Perbedaan antara Traditional dan Pure Functions

- Akses Variabel
Traditional functions dapat mengakses dan mengubah variabel global, sedangkan pure functions tidak.
- Pengaruh terhadap Data
Traditional functions dapat mengubah data, sedangkan pure functions tidak.
- Output
Output dari traditional functions tidak selalu bergantung pada input, sedangkan output dari pure functions selalu bergantung pada input.

3.14 Ciri-ciri Functional Programming

- Data Immutability: Tidak mengubah data di luar scope fungsi.
- First-Class Citizens: Dalam Python, fungsi dapat diperlakukan seperti variabel, dapat disimpan dalam variabel, diteruskan sebagai argumen, atau dikembalikan dari fungsi lain.

3.15 Apa itu Pure Functions?

Pure function adalah fungsi yang tidak mengubah atau mempengaruhi variabel, data, daftar, atau set di luar ruang lingkupnya sendiri.

Contoh: Jika ada daftar dengan ruang lingkup global, pure function tidak dapat menambah atau mengubah daftar tersebut.

3.16 Contoh dan Modifikasi Fungsi Pure

- Contoh Fungsi: Fungsi `add_to` dengan parameter `item` yang menambahkan nilai ke daftar global. Sintaks:

```
def add_to(item):  
    global_list.append(item)
```

Fungsi ini bukan pure function karena mengubah `global_list`.

- Mengubah Menjadi Pure Function: Untuk menjadikannya pure function, fungsi harus menerima daftar sebagai argumen dan mengembalikan daftar baru tanpa memodifikasi yang asli. Sintaks:

```
def add_to_list(lst, item):  
    new_list = lst.copy()  
    new_list.append(item)  
    return new_list
```

Fungsi ini membuat salinan dari daftar asli, menambahkan item ke salinan, dan mengembalikan daftar baru.

3.17 Manfaat Pure Functions

- Konsistensi: Hasil dari pure functions selalu dapat diprediksi, sehingga memudahkan debugging.
- Caching: Pure functions dapat di-cache karena hasilnya selalu sama untuk input yang sama.
- Multi-threading: Pure functions lebih aman dalam program multi-threaded karena tidak mengubah data global, menjaga keandalan data.

3.18 Apa itu Recursion?

Rekursi adalah fungsi yang memanggil dirinya sendiri. Ini menciptakan pola pengulangan.

```
def recursive_function(n):  
    if n == 1:  
        return 1  
    else:  
        return n * recursive_function(n - 1)
```

3.19 Contoh Penggunaan Rekursi

Menghitung faktorial dari sebuah angka.

- Pendekatan Looping: Fungsi menerima integer n dan memeriksa apakah n kurang dari nol. Jika ya, mengembalikan nol. Jika tidak, menghitung faktorial dengan loop.

```
def factorial_loop(n):
    if n < 0:
        return 0
    else:
        factorial = 1
        for i in range(1, n + 1):
            factorial *= i
        return factorial
```

- Pendekatan Rekursif: Fungsi lebih sederhana dan lebih ringkas, memverifikasi apakah n sama dengan satu dan mengembalikan satu jika benar.

```
def factorial_recursive(n):
    if n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)
```

3.20 Kelebihan dan Kekurangan Rekursi

Kelebihan:

- Kode yang lebih rapi dan lebih mudah dibaca.
- Memecah tugas kompleks menjadi sub-masalah yang lebih mudah.

Kekurangan:

- Logika rekursi bisa lebih sulit diikuti.
- Memori yang digunakan bisa lebih banyak dan terkadang tidak efisien.
- Sulit untuk melakukan debugging.

3.21 Membalikkan String dalam Python

Metode 1: Menggunakan Fungsi Slice

- Slice Function: Fungsi yang digunakan untuk mengambil bagian dari string. Sintaksnya adalah

```
string[start:stop:step]
```

Slice function memungkinkan kita untuk mengambil substring dari string berdasarkan indeks yang ditentukan. Contoh:

```
trial = "reversal"
new_trial = trial[::-1] # Membalikkan string
print(new_trial) # Output: "lasrever"
```

Di sini, `[::-1]` berarti kita mengambil seluruh string dari belakang ke depan.

Metode 2: Menggunakan Rekursi

- Recursion: Proses di mana fungsi memanggil dirinya sendiri.

Rekursi adalah teknik pemrograman di mana fungsi memanggil dirinya sendiri untuk menyelesaikan masalah.

Contoh:

```
def string_reverse(str):
    if len(str) == 0:
        return str
    else:
        return string_reverse(str[1:]) + str[0]

reverse = string_reverse("reversal")
print(reverse) # Output: "lasrever"
```

Fungsi `string_reverse` memanggil dirinya sendiri dengan substring yang lebih kecil hingga mencapai string kosong, kemudian membangun string terbalik.

3.22 Fungsi map

`map` adalah fungsi yang menerapkan fungsi tertentu ke setiap elemen dalam daftar dan mengembalikan objek peta. Sintaks:

```
map(function, iterable)
```

3.23 Fungsi filter

`filter` adalah fungsi yang menerapkan fungsi tertentu ke setiap elemen dalam daftar dan mengembalikan elemen yang memenuhi kondisi. Sintaks:

```
filter_coffee = filter(find_coffee, menu)
for x in filter_coffee:
    print(x)
```

3.24 Perbedaan antara map dan filter

- `map`: Mengembalikan semua objek dalam daftar dan menerapkan fungsi, termasuk nilai `None` jika tidak memenuhi kondisi.
- `filter`: Hanya mengembalikan nilai yang memenuhi kondisi, sehingga tidak ada nilai `None` dalam output.

3.25 Comprehensions

Comprehensions dalam Python adalah cara untuk membuat urutan baru dari urutan yang sudah ada.

Ada empat jenis utama comprehension dalam Python:

- List comprehension
- Dictionary comprehension
- Set comprehension
- Generator comprehension

3.25.1 List Comprehension

Sintaksnya adalah:

```
[ <expression> for x in <sequence> if <condition> ]
```

Penggunaannya seperti berikut:

```
data = [2,3,5,7,11,13,17,19,23,29,31]

# Ex1: List comprehension: updating the same list
data = [x+3 for x in data]
print("Updating the list:", data)

# Ex2: List comprehension: creating a different list with
      updated values
new_data = [x*2 for x in data]
print("Creating new list:", new_data)

# Ex3: With an if-condition: Multiples of four:
fourx = [x for x in new_data if x%4 == 0 ]
print("Divisible by four", fourx)

# Ex4: Alternatively, we can update the list with the if
      condition as well
fourxsub = [x-1 for x in new_data if x%4 == 0 ]
print("Divisible by four minus one:", fourxsub)

# Ex5: Using range function:
nines = [x for x in range(100) if x%9 == 0]
print("Nines:", nines)
```

Outputnya seperti ini:

```
Updating the list:  [5, 6, 8, 10, 14, 16, 20, 22, 26, 32,
                    34]
Creating new list:  [10, 12, 16, 20, 28, 32, 40, 44, 52, 64,
                    68]
```

```
Divisible by four [12, 16, 20, 28, 32, 40, 44, 52, 64, 68]
Divisible by four minus one: [11, 15, 19, 27, 31, 39, 43,
    51, 63, 67]
Nines: [0, 9, 18, 27, 36, 45, 54, 63, 72, 81, 90, 99]
```

Contoh yang diberikan menunjukkan berbagai cara penggunaan list comprehensions untuk memperbarui daftar atau menghasilkan daftar baru. Comprehensions menyediakan cara singkat dan elegan untuk memperbarui urutan data. Seperti yang dapat dilihat, kode yang sama dapat ditulis menggunakan loop for konvensional dan kondisi if else.

```
# List comprehension:
data = [x+3 for x in data]

# Regular for loop:
for x in range(len(data)):
    data[x] = data[x] + 3
```

Pemahaman daftar (list comprehension) dapat menjadi pilihan yang lebih baik setelah Anda terbiasa menggunakannya. Perlu dicatat bahwa konsep yang sama dapat diperluas untuk mencakup kondisi if else yang diperlukan.

Pemahaman daftar (list comprehension) adalah yang paling sering digunakan, tetapi ada jenis lain yang juga dapat membuat kode menjadi praktis dan sederhana. Struktur dan sintaksisnya sangat mirip dengan pemahaman daftar (list comprehension) kecuali untuk jenis data yang digunakan.

3.25.2 Dictionary Comprehension

Sintaksnya seperti ini:

```
dict = { key:value for key, value in <sequence> if <
    condition> }
```

Pemahaman kamus menerima satu atau dua daftar sebagai masukan dan menghasilkan kamus darinya. Saya akan menunjukkan cara melakukannya menggunakan hanya satu daftar dan dengan menggunakan dua daftar.

```
# Using range() function and no input list
usingrange = {x:x*2 for x in range(12)}
print("Using range():", usingrange)

# Lists
months = ["Jan", "Feb", "Mar", "Apr", "May", "June", "July",
    "Aug", "Sept", "Oct", "Nov", "Dec"]
number = [1,2,3,4,5,6,7,8,9,10,11,12]

# Using one input list
numdict = {x:x**2 for x in number}
print("Using one input list to create dict:", numdict)
```

```
# Using two input lists
months_dict = {key:value for (key, value) in zip(number,
    months)}
print("Using two lists:", months_dict)
```

Outputnya seperti ini:

```
Using range(): {0: 0, 1: 2, 2: 4, 3: 6, 4: 8, 5: 10, 6: 12,
    7: 14, 8: 16, 9: 18, 10: 20, 11: 22}
Using one input list to create dict: {1: 1, 2: 4, 3: 9, 4:
    16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100, 11: 121,
    12: 144}
Using two lists: {1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr',
    5: 'May', 6: 'June', 7: 'July', 8: 'Aug', 9: 'Sept', 10:
    'Oct', 11: 'Nov', 12: 'Dec'}
```

Di sini saya menggunakan fungsi zip yang menggabungkan dua daftar. Ketika dua daftar memiliki panjang yang tidak sama, panjang daftar yang lebih pendek menjadi panjang kamus.

3.25.3 Set Comprehension

Set Comprehension berkaitan dengan tipe data set dan sangat mirip dengan list comprehension. Perbedaan utama adalah penggunaan kurung kurawal untuk set daripada kurung siku seperti pada list. Misalnya:

```
set_a = {x for x in range(10,20) if x not in [12,14,16]}
print(set_a)
```

Outputnya berikut:

```
{10, 11, 13, 15, 17, 18, 19}
```

Anda dapat melihat bahwa format kode ini mirip dengan yang saya gunakan dalam list comprehensions. Untuk menunjukkan fleksibilitasnya, saya menggunakan kata kunci "not in" untuk memeriksa nilai-nilai dalam daftar. Outputnya adalah nilai-nilai dalam rentang 10 dan 20 yang tidak terdapat dalam daftar tersebut.

3.25.4 Generator Comprehension

Generator comprehension juga sangat mirip dengan list, dengan perbedaan penggunaan kurung kurawal () alih-alih kurung siku ([]). Mereka juga lebih efisien dalam penggunaan memori dibandingkan dengan list comprehensions. Misalnya:

```
data = [2,3,5,7,11,13,17,19,23,29,31]
gen_obj = (x for x in data)
print(gen_obj)
print(type(gen_obj))
for items in gen_obj:
```

```
print(items, end = "\n")
```

Outputnya:

```
<generator object <genexpr> at 0x102a87d60>  
<class 'generator'>  
2 3 5 7 11 13 17 19 23 29 31
```

Dalam kode di atas, saya membuat objek generator dari kelas generator alih-alih daftar. Elemen-elemen dalam objek iterator ini tidak dapat diakses secara langsung dan memerlukan bantuan loop for, sehingga saya mengiterasi elemen-elemen ini dan mencetaknya.

List comprehensions merupakan perkembangan yang relatif baru, namun hal ini tidak berarti mereka lebih efisien. Comprehensions menjadi populer terutama karena memberikan kode yang lebih bersih dan mudah dibaca, serta kemudahan penggunaan. Mereka juga menawarkan beberapa keuntungan tambahan, seperti kemampuan untuk melakukan penyaringan menggunakan kondisi if else.

List comprehensions juga memungkinkan pengembalian langsung daftar, berbeda dengan fungsi map() yang mengembalikan objek map. Kejelasan inilah yang membuat list comprehensions populer, namun fungsi map() masih dianggap sebagai pilihan yang lebih baik saat digunakan pada urutan data yang lebih besar.

3.26 Konsep Dasar Object Oriented Programming (OOP)

Paradigma berorientasi objek diperkenalkan pada tahun 1960-an oleh Alan Kay. Pada saat itu, paradigma tersebut bukanlah solusi komputasi terbaik mengingat skalabilitas perangkat lunak yang dikembangkan saat itu masih terbatas. Seiring dengan meningkatnya kompleksitas perangkat lunak dan aplikasi dunia nyata, prinsip-prinsip berorientasi objek menjadi solusi yang lebih baik.

- Paradigma Pemrograman: Strategi untuk mengurangi kompleksitas kode dan menentukan alur eksekusi. Contoh paradigma termasuk declarative, procedural, dan OOP.
- Object Oriented Programming Adalah Paradigma yang paling banyak digunakan saat ini, memungkinkan penerjemahan masalah dunia nyata ke dalam kode.

3.27 Komponen Utama OOP

- Class: Blok kode logis yang berisi atribut dan perilaku. Didefinisikan dengan kata kunci class.

Definisi: Kelas adalah cetak biru untuk membuat objek. Sintaks:

```
class Employee:  
    def __init__(self, position, status):
```

```
self.position = position
self.status = status
```

- Object: Instansi dari kelas. Setiap objek memiliki atribut dan perilaku yang unik.

Definisi: Objek adalah contoh dari kelas yang diciptakan. Sintaks:

```
emp1 = Employee("Shift_Lead", "Full-Time")
```

- Method: Fungsi yang didefinisikan di dalam kelas yang menentukan perilaku objek.

Definisi: Metode adalah fungsi yang beroperasi pada objek. Sintaks:

```
def intro(self):
    return f"I am a {self.position}."
```

3.28 Konsep Kunci dalam OOP

- Inheritance:

Proses membuat kelas baru dari kelas yang sudah ada. Kelas baru (subclass) mewarisi atribut dan metode dari kelas yang ada (superclass).

```
class Parent:
    Members of the parent class

class Child(Parent):
    Inherited members from parent class
    Additional members of the child class
```

Seiring dengan semakin rumitnya struktur pewarisan, Python mengikuti sesuatu yang disebut **Method Resolution Order (MRO)** yang menentukan alur eksekusi. MRO adalah kumpulan aturan, atau algoritma, yang digunakan Python untuk menerapkan **monotonicity**, yang merujuk pada urutan atau urutan di mana interpreter akan mencari variabel dan fungsi untuk diimplementasikan. Hal ini juga membantu dalam menentukan ruang lingkup anggota-anggota yang berbeda dari kelas yang diberikan.

- Polymorphism:

Kemampuan fungsi untuk beroperasi dengan cara yang berbeda tergantung pada objek. Fungsi dapat memiliki banyak bentuk. Operator + yang berfungsi sebagai penjumlahan untuk angka dan penggabungan untuk string.

Polimorfisme merujuk pada sesuatu yang dapat memiliki banyak bentuk. Dalam hal ini, suatu objek tertentu. Ingatlah bahwa segala sesuatu dalam Python secara inheren adalah objek, jadi ketika saya berbicara tentang

polimorfisme, itu bisa berupa operator, metode, atau objek apa pun dari suatu kelas. Saya dapat menjelaskan contoh polimorfisme menggunakan fungsi bawaan dan operasi, misalnya:

```
string = "poly"
num = 7
sequence = [1,2,3]
new_str = string * 3
new_num = 7 * 3
new_sequence = sequence * 3

print(new_str, new_num, new_sequence)
```

Outputnya akan menjadi:

```
polypolypoly 21 [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Dalam hal ini, operator * menunjukkan polimorfisme. Fungsinya berbeda-beda tergantung pada jenis objek yang diterapkan padanya:

Ketika diterapkan pada string, operator ini mengulang string tersebut.

Ketika diterapkan pada bilangan bulat, operator ini melakukan perkalian.

Ketika diterapkan pada daftar, operator ini mengulang daftar tersebut.

Ini merupakan contoh polimorfisme karena operator yang sama (*) berperilaku berbeda-beda tergantung pada jenis objek yang diterapkan padanya.

Contoh lain:

```
string = "poly"
sequence = [1,2,3]
print(len(string))
print(len(sequence))
```

Outputnya adalah:

```
4
3
```

Dalam contoh ini, fungsi len() bersifat polimorfik karena menghitung panjang berbagai jenis objek:

Untuk string "poly", fungsi ini mengembalikan 4 karena string tersebut terdiri dari 4 karakter.

Untuk daftar [1, 2, 3], fungsi ini mengembalikan 3 karena terdapat 3 elemen dalam daftar.

Meskipun len() adalah fungsi yang sama, fungsinya bekerja secara berbeda tergantung pada jenis objek yang diberikan kepadanya.

- Encapsulation:

Mengikat metode dan variabel dalam satu unit untuk mencegah akses langsung. Mengurangi kesalahan dengan membungkus data dan metode dalam kelas.

Ide enkapsulasi adalah untuk menempatkan metode dan variabel di dalam batas-batas suatu unit tertentu. Dalam kasus Python, unit ini disebut kelas. Dan anggota-anggota kelas tersebut menjadi terikat secara lokal pada kelas tersebut. Konsep-konsep ini lebih mudah dipahami dengan lingkup, seperti lingkup global (yang secara sederhana merujuk pada berkas-berkas yang sedang saya kerjakan) dan lingkup lokal (yang merujuk pada metode dan variabel yang 'lokal' pada suatu kelas). Enkapsulasi sehingga membantu dalam menetapkan lingkup-lingkup ini hingga batas tertentu.

Enkapsulasi juga digunakan untuk menyembunyikan data dan representasi internalnya. Istilah untuk hal ini adalah penyembunyian informasi. Python memiliki cara untuk mengatasinya, tetapi implementasinya lebih baik dalam bahasa pemrograman lain seperti Java dan C++. Modifier akses yang diwakili oleh kata kunci seperti public, private, dan protected digunakan untuk penyembunyian informasi. Penggunaan garis bawah tunggal dan ganda untuk tujuan ini di Python merupakan pengganti praktik tersebut. Misalnya, mari kita lihat contoh anggota yang dilindungi di Python.

```
class Alpha:

    def __init__(self):
        self._a = 2. # Protected member 'a'
        self.__b = 2. # Private member 'b'
```

`self._a` adalah anggota terlindungi dan dapat diakses oleh kelas dan subkelasnya.

Anggota pribadi dalam Python secara konvensional menggunakan dua garis bawah di depan: `__`. `self.__b` adalah anggota pribadi dari kelas Alpha dan hanya dapat diakses dari dalam kelas Alpha.

Perlu dicatat bahwa anggota privat dan terlindungi ini masih dapat diakses dari luar kelas dengan menggunakan metode publik untuk mengaksesnya atau melalui praktik yang dikenal sebagai **name mangling**. Name mangling adalah penggunaan dua tanda underscore di depan dan satu tanda underscore di belakang, misalnya:

```
_class__identifier
```

Class adalah nama kelas dan identifier adalah anggota data yang ingin diakses.

- Abstraction:

Menyembunyikan detail implementasi untuk meningkatkan keamanan data. Menyembunyikan kompleksitas dan hanya menampilkan fitur penting.

Abstraksi dapat dilihat sebagai cara untuk menyembunyikan informasi penting maupun informasi yang tidak perlu dalam blok kode. Intinya, abstraksi dalam Python diimplementasikan melalui kelas dan metode abstrak, yang dapat diimplementasikan dengan mewarisi dari modul yang disebut abc. "abc" di sini singkatan dari abstract base class. Modul ini pertama kali diimpor, lalu digunakan sebagai kelas induk untuk kelas yang menjadi kelas abstrak. Implementasi paling sederhana dapat dilakukan seperti berikut.

```
from abc import ABC,
class ClassName(ABC):
    pass
```

3.29 Kilas Balik Konsep OOP

- Class: Sebuah blueprint untuk membuat objek. Ini mendefinisikan atribut dan metode yang akan dimiliki objek.
- Instance: Objek yang dibuat dari class. Setiap instance memiliki data yang terpisah.
- Object: Entitas yang memiliki atribut dan metode. Dalam Python, hampir semuanya adalah objek.

3.30 Membuat Class

Untuk membuat class, gunakan sintaks berikut:

```
class MyClass:
    pass

class Recipe:
    def __init__(self, dish, items, time):
        self.dish = dish
        self.items = items
        self.time = time
```

pass: Kata kunci yang digunakan sebagai placeholder ketika tidak ada yang perlu dieksekusi.

3.31 Menginstansiasi Class

Untuk membuat instance dari class, gunakan sintaks:

```
myc = MyClass()
```

Di sini, `myc` adalah nama instance yang dapat digunakan untuk mengakses atribut dan metode dari class.

3.32 Mengakses Atribut

Atribut adalah variabel yang dideklarasikan di dalam class. Untuk mengaksesnya, gunakan sintaks:

```
print(myClass.a)
```

Jika `a` adalah atribut dari class, ini akan mencetak nilainya.

3.33 Membuat dan Mengakses Metode

Untuk membuat metode dalam class, gunakan sintaks:

```
def hello(self):  
    print("Hello, world!")
```

`self`: Parameter yang merujuk pada instance saat ini dari class.

Untuk memanggil metode, gunakan sintaks:

```
myc.hello()
```

Ini akan menjalankan metode `hello` dan mencetak "Hello, world!".

3.34 Catatan Lain Terkait Class

- Perubahan pada atribut kelas akan memengaruhi semua instance dari kelas tersebut, karena mereka berbagi atribut kelas yang sama kecuali jika di-override oleh atribut instance. Perhatikan juga penggunaan kata kunci `self` dalam contoh ini. `self` adalah konvensi dalam Python, dan Anda dapat menggunakan kata lain sebagai penggantinya, tetapi sebagai praktik umum, `self` mudah dikenali. `self` dalam contoh ini diteruskan ke dalam metode `cost_evaluation()` karena metode tersebut adalah metode instance dan memudahkan metode tersebut untuk merujuk ke instance `House` mana pun saat metode tersebut dipanggil. Perlu diperhatikan bahwa sejumlah parameter dapat diteruskan ke metode instance ini, tetapi parameter pertama selalu merujuk ke instance kelas tersebut.

```
class House:  
    '''  
    This is a stub for a class representing a house that  
    can be used to create objects and evaluate  
    different metrics that we may require in  
    constructing it.  
    '''  
    num_rooms = 5  
    bathrooms = 2  
    def cost_evaluation(self):
```

```

        print(self.num_rooms)
        pass
        # Functionality to calculate the costs from the
        area of the house

house = House()
print(house.num_rooms)
print(House.num_rooms)

house.num_rooms = 7
print(house.num_rooms)
print(House.num_rooms)

House.num_rooms = 7
print(house.num_rooms)
print(House.num_rooms)

```

Outputnya seperti ini:

```

5
5
7
5
7
7

```

3.35 Code Reusability

Code reusability adalah penggunaan kode yang sudah ada untuk membangun perangkat lunak baru, yang membantu menghemat waktu dan usaha. Dengan menggunakan kelas dan metode yang sama dalam beberapa instance, kita dapat menghasilkan hasil yang berbeda.

3.36 Metode Khusus dalam Python

- `__new__` Method:

Metode ini bertanggung jawab untuk membuat dan mengembalikan objek baru yang kosong. Sintaks:

```

def __new__(cls):
    # kode untuk membuat objek baru

```

- `__init__` Method:

Metode ini digunakan untuk menginisialisasi objek yang baru dibuat dengan nilai-nilai tertentu. Sintaks:

```
def __init__(self, dish, items, time):
    self.dish = dish
    self.items = items
    self.time = time
```

3.37 Instance Variables

- Definisi: Variabel yang dimiliki oleh setiap instance dari sebuah class, menyimpan data spesifik untuk objek tersebut.
- Sintaks: `self.variable_name = value` di dalam metode `__init__`.

3.38 Methods

Definisi: Fungsi yang didefinisikan dalam sebuah class yang menentukan perilaku objek. Contoh:

- Pay Method: Mengubah status pembayaran.

```
def pay(self):
    self.payment = "yes"
```

- Status Method: Menampilkan status pembayaran

```
def status(self):
    if self.payment == "yes":
        return f"{self.name} is paid {self.amount}"
    else:
        return f"{self.name} is not paid yet"
```

3.39 Konsep Inheritance

Inheritance adalah Konsep di mana sebuah kelas (child class) mewarisi atribut dan metode dari kelas lain (parent class). Ini membantu dalam **code reusability** (penggunaan kembali kode).

- Parent Class: Kelas asal yang memberikan atribut dan metode kepada child class. Juga dikenal sebagai superclass atau base class.
- Child Class: Kelas yang mewarisi dari parent class. Juga dikenal sebagai subclass atau derived class.

3.40 Atribut dan Metode

- Atribut: Variabel yang dimiliki oleh kelas. Misalnya, `self.a` dan `self.b`.
- Metode: Fungsi yang didefinisikan dalam kelas. Misalnya, metode untuk meminta cuti dalam child class `Chefs`:

```
def leave_request(self, days):  
    return f"May I take a leave for {days} days?"
```

3.41 Multiple Inheritance

Python juga memungkinkan kita untuk melakukan pewarisan ganda antara kelas-kelas.

Berikut adalah contoh sederhana tentang cara melakukannya.

```
# Example 1  
class A:  
    a = 1  
  
class B:  
    b = 2  
  
class C(A, B):  
    pass  
  
c = C()  
print(c.a, c.b)
```

Outputnya berikut:

```
1 2
```

Pertama, dua kelas bernama A dan B dibuat, lalu variabel a dan b masing-masing diinisialisasi dengan nilai. Kelas baru C kemudian didefinisikan, dan kelas A dan B diteruskan ke dalamnya. Inilah cara pewarisan ganda dilakukan di Python. Urutan kelas penting, tetapi tidak dalam contoh spesifik ini. Saya kemudian membuat objek 'c' dari kelas C. Nilai variabel a dan b dicetak melalui objek c dari kelas C meskipun a dan b tidak ada di dalam kelas C.

Kode di atas adalah contoh pewarisan ganda. Ada juga jenis pewarisan lain yang termasuk dalam kategori pewarisan ganda. Mari kita lihat contoh lainnya.

3.42 Multi-level Inheritance

```
class A:  
    a = 1  
  
class B(A):  
    a = 2  
  
class C(B):  
    pass  
  
c = C()  
print(c.a)
```

Hasilnya adalah 2 karena C mewarisi dari kelas induk langsung C, yaitu B. Meskipun kelas A adalah kelas dasar tingkat atas, kelas C tidak mewarisi langsung dari A, melainkan mewarisi atribut a dari B, dan B memiliki nilai atribut anggota a = 2.

Contoh di atas menunjukkan pewarisan berlevel di mana kelas turunan C mewarisi dari kelas B. Kelas B sendiri adalah kelas turunan dari kelas dasar A. Kelas B di sini merupakan kelas turunan perantara. Dalam kasus ini terdapat tiga level pewarisan, namun dapat diperluas ke banyak level, meskipun mungkin menjadi tidak praktis setelah beberapa waktu.

3.43 Built-in Functions Terkait Class

Ada dua fungsi bawaan yang dapat berguna saat mencoba menemukan hubungan antara kelas dan objek yang berbeda: **issubclass()** dan **isinstance()**.

Fungsi pertama, **issubclass()**, dijelaskan di bawah ini.

```
print(issubclass(A,B))
print(issubclass(B,A))
```

Dua kelas diteruskan sebagai argumen ke fungsi ini dan hasil Boolean dikembalikan.

Ini menunjukkan bagaimana kelas anak (child class) diteruskan sebagai argumen pertama. Untuk menghindari kebingungan, ini dapat dibaca sebagai: “Apakah B adalah subkelas dari A?” Anda dapat melihat hasilnya adalah “Benar” pada kasus kedua di mana kelas anak B adalah subkelas.

Fungsi bawaan lain yang serupa dengan ini adalah **isinstance()**, yang menentukan apakah suatu objek merupakan instance dari suatu kelas. Jadi, jika saya menulis:

```
class A:
    pass
class B(A):
    pass

b = B()
print(isinstance(b,B))
print(isinstance(b,A))
```

Hasil yang akan saya dapatkan adalah “True”.

Sekarang setelah Anda tahu bagaimana kelas dapat diperluas dari kelas lain, mari kita lihat fungsi bawaan lain yang berguna bernama fungsi **super()**.

Fungsi **super()** adalah fungsi bawaan yang dapat dipanggil di dalam kelas turunan dan memberikan akses ke metode dan variabel kelas induk atau kelas saudara. Kelas saudara adalah kelas yang memiliki kelas induk yang sama. Ketika Anda memanggil fungsi **super()**, Anda akan mendapatkan objek yang mewakili kelas induk sebagai hasilnya.

Fungsi **super()** memainkan peran penting dalam pewarisan ganda dan membantu mengarahkan alur eksekusi kode. Fungsi ini membantu dalam mengelola

atau menentukan kontrol dari mana saya dapat mengambil nilai-nilai fungsi dan variabel yang diinginkan.

Jika Anda mengubah sesuatu di dalam kelas induk, perubahan tersebut akan langsung diterapkan di dalam kelas turunan. Hal ini terutama digunakan di tempat-tempat di mana Anda perlu menginisialisasi fungsi-fungsi yang ada di dalam kelas induk juga di dalam kelas turunan. Anda kemudian dapat menambahkan kode tambahan di dalam kelas turunan.

Berikut adalah contohnya.

```
class Fruit():
    def __init__(self, fruit):
        print('Fruit type: ', fruit)

class FruitFlavour(Fruit):
    def __init__(self):
        super().__init__('Apple')
        print('Apple is sweet')

apple = FruitFlavour()
```

Output:

```
Fruit type:  Apple
Apple is sweet
```

Dalam kode di atas, jika saya telah mengomentari baris untuk fungsi `super()`, outputnya adalah:

```
Apple is sweet
```

Hal ini terjadi karena saat Anda menginisialisasi kelas turunan, Anda tidak menginisialisasi kelas dasar bersamanya. Fungsi `super()` membantu Anda mencapai hal ini dan menambahkan inisialisasi kelas dasar bersama dengan kelas turunan.

3.44 Kelas Abstrak (Abstract Class)

Kelas abstrak adalah kelas yang tidak dapat diinstansiasi (tidak bisa dibuat objeknya) dan digunakan sebagai dasar untuk kelas lain. Fungsinya adalah menjamin bahwa semua kelas yang diturunkan dari kelas ini memiliki metode tertentu yang harus diimplementasikan.

3.45 Metode Abstrak (Abstract Method)

Metode abstrak adalah metode yang didefinisikan dalam kelas abstrak tanpa implementasi. Kelas yang diturunkan harus memberikan implementasi untuk metode ini. Fungsinya adalah memastikan bahwa setiap kelas turunan memiliki metode yang sama, meskipun cara implementasinya bisa berbeda.

3.46 Sintaks untuk Mendefinisikan Kelas Abstrak dan Metode Abstrak

```
from abc import ABC, abstractmethod

# Mendefinisikan kelas abstrak
class SomeAbstractClass(ABC):
    @abstractmethod
    def some_method(self):
        pass
```

3.46.1 Contoh Implementasi

- Kelas Abstrak: Misalnya, kita memiliki kelas `Vehicle` yang merupakan kelas abstrak.
- Kelas Turunan: Kelas `Car` dan `Boat` yang diturunkan dari `Vehicle` harus mengimplementasikan metode `turn_on_engine`.

```
class Vehicle(ABC):
    @abstractmethod
    def turn_on_engine(self):
        pass

class Car(Vehicle):
    def turn_on_engine(self):
        return "Mesin mobil dinyalakan!"

class Boat(Vehicle):
    def turn_on_engine(self):
        return "Mesin perahu dinyalakan!"
```

3.47 Konsep Dasar MRO

MRO adalah aturan yang menentukan urutan pencarian metode atau atribut dalam hierarki kelas. Ini membantu dalam menentukan dari mana metode atau atribut tersebut berasal.

Linearization: Proses di mana MRO mengurutkan kelas untuk pencarian metode, mengikuti aturan tertentu.

3.47.1 Tipe-tipe Inheritance

- Simple Inheritance: Kelas anak mewarisi dari satu kelas induk.
- Multiple Inheritance: Kelas anak mewarisi dari lebih dari satu kelas induk.

- Multilevel Inheritance: Inheritance yang terjadi pada beberapa level, di mana kelas anak mewarisi dari kelas induk yang juga merupakan kelas anak dari kelas lain.
- Hierarchical Inheritance: Beberapa kelas anak mewarisi dari satu kelas induk yang sama.
- Hybrid Inheritance: Kombinasi dari beberapa tipe inheritance di atas.

3.48 Aturan MRO

- Default Order: Di Python, urutan pencarian adalah dari bawah ke atas dan dari kiri ke kanan dalam struktur pohon kelas.
- Contoh: Jika kelas Z mewarisi dari kelas X dan Y, urutan MRO-nya adalah Z, Y, X.

3.49 Algoritma MRO

- C3 Linearization: Algoritma yang digunakan untuk menentukan MRO di Python versi 3 dan seterusnya.
- Monotonicity: Properti yang tidak membolehkan melewati kelas induk langsung saat mewarisi.
- Inheritance Graph: Kelas induk hanya dikunjungi setelah metode lokal kelas telah dikunjungi.

3.50 Menemukan MRO

- MRO Function: Untuk menemukan urutan MRO dari sebuah kelas, gunakan fungsi `mro()`.

```
class C(B, A):
    pass

print(C.mro())
```

- Help Function: Menggunakan fungsi `help()` untuk mendapatkan informasi lebih detail tentang MRO dan atribut kelas.

```
help(C)
```

3.51 Eksperimen MRO

3.51.1 Contoh 1

```
# Example 1
class A:
    def a(self):
        return "Function inside A"

class B:
    def a(self):
        return "Function inside B"

class C(B,A):
    pass

# Driver code
c = C()
print(c.a())
```

Output:

```
Function inside B
```

Kelas C mewarisi dari kelas B dan A. Jika saya tidak menemukan fungsi a() di dalam kelas C, saya harus mencari kelas B dan A, dan penting untuk melakukannya dalam urutan tersebut.

Sekarang saya akan menambahkan satu tingkat lagi ke dalam ini dan mencatat hasilnya.

3.51.2 Contoh 2

```
class A:
    def b(self):
        return "Function inside A"

class B:
    def b(self):
        return "Function inside B"

class C(A, B):
    def b(self):
        return "Function inside C"
    pass

class D(C):
    pass

d = D()
print(d.b())
```

Output:

```
Function inside C
```

Kelas D mewarisi dari kelas C, yang pada gilirannya mewarisi dari kelas A dan B. Kelas D mengakses kelas induk langsung dari kelas D, yaitu kelas C, dan menentukan nilai variabel tersebut setelah menemukannya di kelas induk tersebut.

Sekarang, katakanlah saya mengomentari deklarasi di dalam kelas C.

```
# def b(self):  
#     return "Function inside C"
```

Dan ganti dengan kata kunci pass agar kode tetap berfungsi.

Karena tidak ada nilai yang terdapat di dalam kelas C, panggilan fungsi di atas akan diarahkan ke A. Hal ini karena kelas C akan mengacu pada kelas A sebagai yang memiliki prioritas lebih tinggi saat mewarisi.

Sekarang mari kita ambil contoh lain dari skenario serupa.

3.51.3 Contoh 3

```
class A:  
    def c(self):  
        return "Function inside A"  
  
class B:  
    def c(self):  
        return "Function inside B"  
  
class C(A, B):  
    def c(self):  
        return "Function inside C"  
  
class D(A, C):  
    pass  
  
d = D()  
print(d.c())
```

Output:

```
Traceback (most recent call last):  
  File "/Users/intropython/PycharmProjects/practicePython/  
    inherit.py", line 10, in <module>  
    class D(A, C):  
TypeError: Cannot create a consistent method resolution  
order (MRO) for bases A, C
```

Perhatikan bahwa hal ini menyebabkan kesalahan. Dalam kode di atas, kelas D mewarisi dari kelas A dan kelas C.

Kelas C adalah kelas induk langsungnya, tetapi karena ini adalah pewarisan ganda, aturannya lebih rumit dan juga harus memeriksa kelas-kelas yang diteruskan kepadanya untuk menentukan prioritas.

Dalam kasus ini, kelas D tidak dapat menentukan urutan yang harus diikuti saat menentukan nilai untuk variabel dalam kasus di mana variabel tersebut tidak ada dalam kelas objek yang diberikan.

Hal ini menyebabkan TypeError karena tidak dapat membuat Method Resolution Order (MRO). MRO adalah cara Python untuk menentukan urutan prioritas kelas saat menangani pewarisan.

Mari kita lihat contoh terakhir.

3.51.4 Contoh 4

```
class A:
    def d(self):
        return "Function inside A"

class B:
    def d(self):
        return "Function inside B"

class C:
    def d(self):
        return "Function inside C"

class D(A, B):
    def d(self):
        return "Function inside D"

class E(B, C):
    def d(self):
        return "Function inside E"

class F(E,D,C):
    pass

f = F()
print(f.d())
print(F.mro())
```

Output:

```
Function inside E
[<class '__main__.F'>, <class '__main__.E'>, <class '__main__.D'>, <class '__main__.A'>, <class '__main__.B'>, <class '__main__.C'>, <class 'object'>]
```

Kode di sini cukup sederhana. Kelas F secara langsung mewarisi dari kelas induk langsungnya dan kelas pertama yang diteruskan kepadanya. Baris kedua

kemudian menunjukkan hasil dari fungsi `mro()`.

Contoh-contoh dalam bacaan ini menunjukkan bagaimana kode yang menggunakan pewarisan ganda dapat menjadi rumit dan berantakan dengan sangat cepat. Pewarisan ganda, dengan semua keunggulan dan fleksibilitas yang ditawarkannya, sebaiknya hanya digunakan setelah Anda menguasai Python sebagai bahasa pemrograman dengan baik untuk menghindari pembuatan 'kode spaghetti' yang sulit dipahami dan diperbarui.

4 Modul, Paket, Pustaka, and Alat-alat